Cryptography Main (Updated Edition 5)

Cryptography Main (Updated Edition 5)

Table of Contents

Cryptography Main (Updated Edition 5)

A single, deduplicated, LLM-friendly compilation covering the history and foundations of cryptography, classical cryptanalysis, the formal/mathematical basis of modern cryptography, symmetric and public-key primitives, protocols, advanced constructions, implementation security, and the socio-political dimensions of the field.

Update note (Edition 5): This edition was produced by checking the prior compilation ("Cryptography Main 4") against five supplied source files: `Cryptography_Main4.pdf` (the prior edition itself), `kerckhoffs.pdf` (a direct scan of both parts of Auguste Kerckhoffs' 1883 "La Cryptographie Militaire"), `KerckhoffsPrinciple-CryptoDictionnary.docx` (Fabien Petitcolas' dictionary entry on Kerckhoffs' Principle), `Oded_Goldreich-Foundations_of_Cryptography__Volume_2__Basic_Applications_2009_.pdf` (Cambridge University Press, 2004/2009 — this is genuinely **Volume 2**, covering Encryption Schemes, Digital Signatures and Message Authentication, and General Cryptographic Protocols, as distinct from the Volume 1 file reviewed in Edition 4), and `Understanding_Cryptography_by_Cristof_Paar__Jan_Pelzl__and_Tim_Güneysu.pdf` (Paar, Pelzl & Güneysu, *Understanding Cryptography*, 2nd ed., Springer 2024).

Findings for each file:

1. `kerckhoffs.pdf` — a 71-page scan containing both Part I (Journal des Sciences Militaires, Jan. 1883, pp. 5–38) and Part II (Feb. 1883, pp. 161–191) of Kerckhoffs' original French article. This is the same source material (Part I and Part II) already fully incorporated into §1 of the prior edition, confirmed section-by-section against the historical narrative, the six principles, the cipher taxonomy, the cryptanalysis techniques (key-length determination, symmetry of position, the Havas/Reuters telegram walkthrough, the triple-key curiosity), the coded-dictionary material, the cryptograph survey, and the closing du Carlet maxim. No changes needed.
2. `KerckhoffsPrinciple-CryptoDictionnary.docx` (Petitcolas' dictionary entry) — a shorter digest of the same material, already fully incorporated (and explicitly cited as a source) in the prior edition. No changes needed.
3. `Cryptography_Main4.pdf` — the prior edition itself; used as the base document for this edition.
4. `Oded_Goldreich-Foundations_of_Cryptography__Volume_2__Basic_Applications_2009_.pdf` (Goldreich, *Foundations of Cryptography, Volume 2: Basic Applications*, Cambridge University Press) — a full, currently-copyrighted 449-page textbook, not a research digest. Per copyright limits, this edition does not ingest or closely paraphrase its detailed expositions, proofs, or worked examples. Its table of contents and chapter structure (Encryption Schemes; Digital Signatures and Message Authentication; General Cryptographic Protocols) was reviewed at the topic level against the prior edition's coverage. Most of its core material (semantic security, IND-CPA/CCA, private- and public-key encryption, EU-CMA signatures, one-time signatures, Merkle-tree signatures, fail-stop signatures, Yao/GMW-style secure multiparty computation) was already present from Edition 4's own reading of Katz–Lindell-style definitions and the MPC literature. A small number of named concepts covered by the book but entirely absent from the prior edition — non-malleable encryption, universal one-way hash functions (and the one-time-to-many-time signature paradigm they enable), unique signatures, strong (super-secure) unforgeability, and incremental signatures — have been added in this edition as brief, original-wording, standard-reference-level entries (new §12.1a). No text, proofs, or examples are reproduced from the book itself.
5. `Understanding_Cryptography_by_Cristof_Paar__Jan_Pelzl__and_Tim_Güneysu.pdf` (Paar, Pelzl & Güneysu, *Understanding Cryptography*, 2nd ed., Springer 2024) — likewise a full, currently-copyrighted 555-page textbook (14 chapters plus references and index), not a digest. Per copyright limits, this edition does not ingest or closely paraphrase its text, figures, or worked numerical examples. Its chapter and section structure was reviewed at the topic level only. The great majority of its subject matter (stream ciphers, DES/AES, block cipher modes, RSA, Diffie–Hellman, ECC, digital signatures, hash functions, MACs, PQC, key management) was already covered by the prior edition in independently-written, condensed reference form. A handful of named concepts, algorithms, and results entirely absent from the prior edition have been added as brief original entries: Shannon's confusion/diffusion design principles (§3.1); the OFB and CFB block-cipher modes and the key-whitening/DESX construction (§3.1); Euler's totient function, Fermat's Little Theorem, and Euler's Theorem as the number-theoretic basis of RSA correctness (§2.1); the ElGamal encryption scheme and the ElGamal signature scheme (§6.2, §6.4); and the Needham–Schroeder protocol for symmetric-key-based key establishment (§7.6). No text, tables, or worked examples are reproduced from the book itself.

Everything else previously verified in Edition 4 (the Kerckhoffs Part I/II historical material, the Random Oracle Model treatment, the Goldreich Volume 1 additions, the Handbook of Applied Cryptography additions, the ZKP/SNARK/STARK/Bulletproofs material, the IKEv2/ECC/side-channel material, and the socio-political material) remains unchanged and is carried forward as-is.

Note on source coverage: as in prior editions, a further file (`*_Jonathan_Katz.pdf*`, presumed Katz & Lindell) was not independently re-verified in this edition either, since it was not supplied this round. §2.5 and §22's treatment of that book is carried forward unchanged, with

the same caveat.

Sources merged (nothing repeated twice):

1. Fabien Petitcolas' "Kerckhoffs Principle" dictionary entry + Auguste Kerckhoffs' original 1883 article "La Cryptographie Militaire" (Journal des sciences militaires) — historical foundations.
2. The full French original text of "La Cryptographie Militaire," Part I (scanned/OCR'd PDF) — used to fill in taxonomy, biographical detail, and extended historical narrative and worked examples not present in the digest (incorporated in Edition 2).
3. The full French original text of "La Cryptographie Militaire," Part II, pp. 161-191 (scanned/OCR'd PDF, crypto_militaire_2.pdf) — used in Edition 3 to fill in the worked cryptanalysis examples, device history, and coded-dictionary history that the Petitcolas digest had only summarized.
4. A prior compiled "Cryptography Master Reference" covering mathematical foundations through modern primitives, protocols, and socio-political context.
5. Jonathan Katz & Yehuda Lindell, Introduction to Modern Cryptography (2nd ed., CRC Press) — used only for its topic structure (formal security definitions, the provable-security paradigm) summarized in the compiler's own words; no text is reproduced from the book, in line with copyright limits. (Not independently verified in Edition 3, 4, or 5 — see notes above.)
6. "Cryptography 2: Advanced Theoretical Foundations, Protocol Architecture, and Socio-Political Dimensions" (a companion research digest) — used in Edition 3 to add the Random Oracle Model formal treatment in §2.6; its other content was cross-checked against the existing compilation and found to already be covered.
7. Cryptography_3_Compendium.pdf ("The Complete Cryptography Compendium") — re-verified in Edition 4; confirmed already fully incorporated.
8. Cryptography.pdf ("Find Deep Cryptography YouTube Videos," OCR'd) — re-verified in Edition 4; confirmed already fully incorporated into §19.2.
9. Oded Goldreich, Foundations of Cryptography, Volume 1: Basic Tools (Cambridge University Press, 2001) — reviewed at the topic-structure level only in Edition 4, per copyright limits; a small number of named concepts absent from prior editions added in original wording. No text reproduced.
10. Menezes, van Oorschot & Vanstone, Handbook of Applied Cryptography (CRC Press) — reviewed at the topic-structure level only in Edition 4, per copyright limits; a small number of named algorithms/protocols/historical items absent from prior editions added in original wording. No text reproduced.
11. Oded Goldreich, Foundations of Cryptography, Volume 2: Basic Applications (Cambridge University Press, 2004/2009) — reviewed at the topic-structure level only in Edition 5, per copyright limits; a small number of named concepts absent from prior editions added in original wording (see Update Note above). No text reproduced.
12. Christof Paar, Jan Pelzl & Tim Güneysu, Understanding Cryptography: From Established Symmetric and Asymmetric Ciphers to Post-Quantum Algorithms (2nd ed., Springer, 2024) — reviewed at the topic-structure level only in Edition 5, per copyright limits; a small number of named concepts, modes, and protocols absent from prior editions added in original wording (see Update Note above). No text reproduced.

1. Historical Foundations — Kerckhoffs and Classical Cryptography

1.0 Who Was Kerckhoffs?

Auguste Kerckhoffs (1835–1903) was a Dutch-born linguist, trained at the University of Liège, who taught German at the École des Hautes Études Commerciales and the École Arago in Paris. Alongside a side interest in constructed languages (he supported Volapük), he had a

serious interest in cryptography. In January and February 1883 he published a two-part article, “La Cryptographie Militaire,” in the Journal des sciences militaires (vol. IX, pp. 5–38 and pp. 161–191), surveying the cryptographic methods of his day and setting out design principles for military ciphers. The article opens with an epigraph from General Lewal’s Études de guerre, calling cryptography “a powerful auxiliary of military tactics.” This article is the origin of what is now called Kerckhoffs’ Principle.

1.0.1 Cryptography Before Kerckhoffs: A Historical Sketch

Kerckhoffs opened his article with a historical survey. He traced cryptography (or “the Art of ciphering”) to antiquity, noting it was practiced by the Chinese, Persians, and Carthaginians, taught in the tactical schools of Greece, and held in high esteem by the most illustrious Roman generals. Beyond the scytale of the Lacedaemonians and the tricks recorded by Aeneas Tacticus, he could find only scattered references in Polybius, Plutarch, Dio Cassius, Suetonius, Aulus Gellius, Isidore, and Julius Africanus — evidence, he argued, of how thin the surviving record of ancient cryptographic practice really is.

During the Middle Ages, Kerckhoffs observed, cryptography was cultivated mainly by monks and cabalists, and even then inventors were usually more interested in disguising the existence of a message (steganography) than in building genuinely hard-to-break ciphers. He notes that even in the 17th century, English courts treated the mere fact of having corresponded in cipher as an aggravating circumstance: in the notorious poisoning trial of the Count of Somerset, Chancellor Bacon cited the accused’s habit of writing to friends in cipher as a serious charge against him.

Kerckhoffs then traces cryptography’s rise as a genuine “art” from the Renaissance onward: Henri IV used a cipher for his intimate correspondence with the Landgrave of Hesse. When Henri IV intercepted ciphered letters between members of the Catholic League and the Spanish government, he had the mathematician François Viète find the key; Viète succeeded, letting the king monitor his enemies’ intrigues for nearly two years. Under Richelieu, the art of deciphering rose almost to the level of a state science; according to Beausobre, the Foreign Ministry even ran an academy where it was taught.

Kerckhoffs could find no clear trace of cryptographic correspondence in the French army as early as the 16th century, but by the 18th century, orders to generals commanding on the frontiers or in enemy territory were transmitted only in cipher.

During the First Empire, generals used two keys: the grand chiffre and the petit chiffre or chiffre banal. Baron Fain, Napoleon I’s secretary, reports that the Emperor maintained ciphered correspondence during the Russian campaign. During the Peninsular War, a Spaniard stole the cipher used by Marshal Suchet and used it to help retake Mequinenza and Lérida.

1.0.2 The State of the Question in 1883

Kerckhoffs contrasted French practice with German doctrine: German military schools taught cryptanalysis itself as a subject, not just enciphering. Article 32 of the French regulation of 19 January 1874 stated that military dispatches should, as far as possible, be ciphered — yet Kerckhoffs noted that ciphered correspondence in the French army was still largely confined to commanders-in-chief. Cautionary anecdotes he offered include:

General Bardin’s account of the chaotic 1814 campaign, when Marshals Feltre and Berthier sent orders in plain, unciphered French and most fell into enemy hands.

A Russo-Turkish War episode: a ciphered dispatch reached the Ministry of War on a Sunday while the officer holding the key was absent; a staff officer, Captain Henri Berthaut (who happened to be the Minister’s own son), decrypted it without the key within hours.

German press obituaries (1879) for Captain Max Hering crediting the 1870 discovery of the Seine submarine cable partly to the absence of a secure cipher between Paris and the provincial armies during the siege.

Kerckhoffs closes this section by invoking Voltaire's jab in the Dictionnaire philosophique and a similar line from the Earl of Clarendon dismissing self-proclaimed decipherers as "mountebanks," and cites "citizen" Dlandol's 1793 pamphlet Le Contr'espion as a precedent for publishing France's own cryptographic weaknesses.

1.0.3 Practical Transmission Rules

Kerckhoffs gave concrete formatting rules for any single-key cipher intended for real telegraphic use: (1) word boundaries should not be indicated in the ciphertext; (2) doubled letters should be suppressed; (3) no capital letters, accents, or punctuation should be used. He also recommended breaking cryptograms into fixed groups of four or five characters, matching the five-letter groups by which telegraph administrations billed secret dispatches.

1.1 The Six Principles for a Military Cipher

Kerckhoffs argued that a cipher meant for indefinite, ongoing military use must satisfy six conditions:

1. Practical unbreakability — unbreakable in practice, if not in a strict mathematical sense.
2. No requirement of secrecy — must not depend on being secret, and should cause no harm if it falls into enemy hands. (This is the modern "Kerckhoffs' Principle.")
3. Memorable, changeable keys — short enough to memorize, easy to change.
4. Telegraph-compatible.
5. Portable, single-operator.
6. Simple to use — no long list of rules under stressful conditions.

Kerckhoffs treated (4)–(6) as uncontroversial; his real argument concerned (1)–(3), especially Principle 2. A secret system can only be entrusted to a small number of people, restricting its use to high commanders; it is vulnerable at every "engagement" where it might be captured; and messages must often stay secret for far longer than "a few hours." His policy recommendation: the War Ministry should abandon secret methods entirely and adopt only a system that could be taught openly and even copied by rival nations, because true security should come only from the key.

Shannon's Maxim. Claude Shannon later gave the information-theoretic grounding for this, restating it as "the enemy knows the system" — treated as functionally equivalent to Kerckhoffs' Principle, and standing in direct opposition to security through obscurity.

1.2 Cipher Taxonomy (as surveyed by Kerckhoffs, 1883)

Kerckhoffs organized 19th-century ciphers into three families:

A. Transposition ciphers. Rearrange the letters of the plaintext without changing them. Includes simple columnar transposition (Kerckhoffs' worked example enciphers "Une attaque simulee aura lieu demain matin a quatre heures" via two column reorderings), double transposition (his worked example on Russian nihilist ciphers, enciphering "Vous etes invite a vous trouver ce soir…" through two 9x9 grids — he calls reusing the same keyword for both column and row transposition "a very serious mistake"), and grille ciphers (Cardano's cut-out grid, Fleissner's 1881 rotating refinement).

B. Substitution ("interversion") ciphers. Single-alphabet (monoalphabetic) ciphers, credited to Caesar, Augustus, Trithemius, Porta, Vigenère, Bacon, Hermann, and Mirabeau, all judged breakable "with equal ease" except Hermann's. Worked examples include a Champigny-keyed substitution alphabet and the Klinglin correspondence seized at Offenbourg. Polyalphabetic (double-key) ciphers, including Porta's cipher (1563), Vigenère's tableau ("le chiffre indéchiffrable"), the Saint-Cyr system, the Beaufort cipher, the Gronsfeld cipher, and the variable-key "running" system with a stop letter.

C. Coded dictionaries (“dictionnaires chiffrés” / nomenclators). Whole words, syllables, or phrases replaced by numeric codes from a codebook, sometimes with an added permutation-plus-addition step (Brunswick and Gallian’s “double-combination” codebooks) to further disguise the raw code numbers. Kerckhoffs judged these among the most practically secure systems of the era in principle, but criticized them heavily because they depend on absolute physical secrecy of the codebook (violating Principle 2), and because a single mistyped digit can silently and completely change the meaning of an order.

Historical background (expanded in Ed. 3, from the Part II scan). Kerckhoffs traces the lineage of the French military codebook back to a 2 May 1815 letter from the Minister of War, Davout, to his colleague at Foreign Affairs, requesting two ciphers of this kind for military correspondence (as reported by General Lewal in the Journal des Sciences Militaires); these “tables chiffrantes” evolved into the coded dictionary used by the army in Kerckhoffs’ own day. He then surveys the civilian codebook tradition that followed: Brachet’s 1850 Dictionnaire chiffré (each word represented by an invariable five-digit number); Sittler’s 1868 Dictionnaire abréviatif chiffré, a fifty-page vocabulary of 100 words per page (numbered 00–99) where the page number is added by hand under a separate convention — a scheme Kerckhoffs praises for its design but faults for never addressing what happens if the book itself is captured; and the double-combination dictionaries of Brunswick (Dictionnaire pour la correspondance télégraphique secrète, 1868, representing words and letter-pairs as four-digit groups 0000–9999) and Gallian (Dictionnaire télégraphique économique et secret, 1874, using three-letter groups instead). He notes the same double-combination principle underlies the German dictionaries of Niethe (Berlin, 1877) and Walert (Winterthur, 1877) and Bolton’s English equivalent, plus a 20,000-word dictionary published by M. Louis, director of the Journal des Postes. In a footnote he also mentions a related but distinct trick — replacing important words with unrelated cover words — citing papers seized from Prince Louis-Napoléon’s 1831 Bonapartist plot in which Queen Hortense was code-named “M. Antoine,” Louis-Napoléon himself “Mme Charles,” England “Mme Lirson,” the Bonapartists “Mme Gock,” and the army “Mlle Amélie.”

The Brunswick double-combination mechanism, worked (new in Ed. 3). Brunswick’s method first permutes the digits of a codebook number according to an agreed formula, then adds (or subtracts) a second agreed number; the added/subtracted number, together with the permutation formula, forms the key. Kerckhoffs’ example: the codebook number for avancez is 2143; permuted to 2134 (one of twelve possible orderings), then increased by the conventional number 214, giving a transmitted group of 2348. He shows that once an analyst possesses the codebook itself, recovering the key from just two known plaintext-to-ciphertext-group pairs is straightforward — illustrated with a fully worked reverse-engineering example: given that intercepted groups 9645 and 7457 are suspected to mean colonel and régiment (whose codebook numbers are 4913 and 2734), comparing digit-by-digit reveals a three-digit key, an addition (not subtraction), and a permutation that moves the second digit to the first position; subtracting all six possible permutations of 413 and 234 from 643 and 457 respectively until a common difference emerges yields key number 214 and permutation formula b a d c.

Operational failures (new in Ed. 3). Kerckhoffs catalogs several real incidents illustrating the fragility of codebook secrecy: on 8 January 1871, a cryptogram from the Prussian king’s headquarters reached General von Werder, who could not decode it immediately because the codebook was locked in a valise on a distant carriage; during the Russo-Turkish War, in September 1877, Selim Pasha (deputy political chief to Mehmet Ali) accidentally took the decoding book with him on a short absence, leaving the commander-in-chief unable to read dispatches for several days; a simple transcription error once led the Italian ministry to believe the Austro-Hungarian statesman Count Andrássy was expected in St. Petersburg, when the garbled digit group actually named a minor embassy attaché; and during the 1870 war, roughly half of the dispatches sent by military authorities to prefects contained illegible or

corrupted sections, a problem repeated during the Tunisia campaign. Kerckhoffs concludes that coded dictionaries are only practical for chancelleries, whose trained staff and inviolable diplomatic status let them verify transmissions carefully — not for the battlefield.

1.3 Cryptographic Devices ("Cryptographes")

Kerckhoffs reviewed the mechanical cipher devices of his era:

Ancient devices: the Spartan scytale, Aeneas Tacticus's perforated board ("planchette d'Énée," pierced with 24 holes representing the alphabet, through which a string was threaded to indicate letter order), "Kessler's barrel" (1616).

Kircher's Arca glottotactica (new in Ed. 3): Athanasius Kircher's mechanical device, a kind of mobile catalogue in which words were sorted according to their corresponding alphabet letters (cited by Kerckhoffs via Schott's Schola steganographica).

Chappe's aerial telegraph and the Morse telegraph (new in Ed. 3): Kerckhoffs explicitly classes both of these as "true cryptographs" in the same lineage as the mechanical devices above.

Porta's disk cipher (1563) — the origin of the rotating-disk substitution idea, later reused in Wheatstone's cryptograph and various dial-based devices sold during the Italian war of 1859.

Mouilleron's device — a mechanism mounted on a writing desk, described by Kerckhoffs as overly complicated for the security it provides. He demonstrates (worked example, new detail in Ed. 3) that despite its bulk it reduces mathematically to an ordinary Vigenère cipher: the author's own worked example, "La victoire est à nous" enciphered with the key paris on the Mouilleron apparatus, produces exactly the same cryptogram as enciphering the same plaintext on the standard Vigenère table with the key kzirh — kzirh being simply paris written with the alphabet reversed.

Vinay and Gaussin's device — a combined printer/encryption mechanism, more portable than Mouilleron's but still too bulky for field use, and, per Kerckhoffs, of no cryptographic value whatsoever.

Rondepierre's "phyrographe" — an ivory-rod transposition device: vertical columns are represented by ivory rods bearing letter divisions; the rods are transposed per a numeric formula, the message is pencilled onto them, then they are restored to normal order and the letters copied off in their new sequence. Kerckhoffs judged it fairly practical but only relatively secure.

Silas's cryptograph — devised by a former attaché to the French embassy in Vienna; Kerckhoffs calls it serious, well-designed work, but too complex for wartime use.

Wheatstone's cryptograph — judged the best of the newer devices for simplicity. Kerckhoffs reproduces the mechanism in detail (expanded in Ed. 3) from the 1867 Exposition Universelle military commission report: a keyword such as projectile is written with its letters spaced out, and the unused alphabet letters are written beneath it in normal order across the remaining columns; reading the letters down each vertical column in turn yields a scrambled "conventional alphabet," which is printed on an inner cardboard disk sitting concentrically over an outer metal dial bearing the normal alphabet (plus a word-stop symbol, "+", not used in the cipher itself). Two hands move simultaneously over the double dial at different, linked speeds; advancing the outer (plaintext) hand to a letter causes the inner (ciphertext) hand to stop on the corresponding cipher letter, so the device encodes both an alphabet permutation and a starting offset as its two keys. Kerckhoffs judged it fully breakable in practice — once the cycle length before the alphabet arrangement repeats is known (e.g., after the 26th or 30th letter), the cryptogram can be split into columns of that length and attacked exactly like any other polyalphabetic cipher — and, like all secret-device schemes, it still requires total secrecy of the device to have any value at all, since capturing the apparatus lets an adversary

recover the starting point after only a little trial and error on the first few letters. (He credits Wheatstone separately with having deciphered, in 1858, several interesting letters of Charles I written entirely in Arabic numeral cipher.)

Kerckhoffs closes §1.3 by noting (new in Ed. 3) that a new cryptograph had just been presented to the French Military Telegraphy Commission at the time of writing, which he believed satisfied all of his desiderata — complete indecipherability, simplicity, and no requirement of secrecy — but declined to describe further "for reasons of high propriety."

1.4 Breaking Ciphers — Kerckhoffs' Cryptanalysis Techniques

Kerckhoffs laid out concrete, step-by-step cryptanalytic techniques current in the 1880s — an early practical treatise that predates and, in places, anticipates the modern Kasiski examination (published by Major Kasiski in 1863, which Kerckhoffs considered unnecessarily complicated). He opens Part II by noting that, to his knowledge, only Kasiski had previously published any serious attempt at breaking double-key (polyalphabetic) ciphers; everything in Porta, Cospi, Breithaupt, 's Gravesande, Thicknesse, Klüber, Lacroix, Vesin, and Joliet applied only to single-key systems, and mostly assumed word boundaries were visible in the ciphertext (new in Ed. 3). He quotes a suspicious aside from Cospi, secretary to the Grand Duke of Tuscany, distinguishing "simple" from "composed" ciphers and declining to discuss the latter as "almost impossible to decipher" — which Kerckhoffs takes as evidence that decipherers have long concealed their methods from outsiders, since it seems unlikely Cospi could not break the systems Porta and Vigenère had published half a century earlier.

Letter-frequency analysis: in French, roughly one letter in five is "e"; Kerckhoffs gives full letter-frequency tables from real French military correspondence (E=185, S=88, R=78, I=74, A=72 per 1,000 letters, etc.) and notes the most frequent letter differs by language — E for French/English/German, O for Spanish, A for Russian, E/I for Italian. He also reports a consonant/vowel split of 560:440 per 1,000 letters, and gives the most frequent letters in German (E, N, I, R, S, T, U, D, A, H) and English (E, T, A, O, N, I, R, S, H, D, L). Beyond single-letter frequency, the most common French bigrams built around e are es and en, followed by se, te, et, de, me, el, em, le.

Breaking monoalphabetic substitution: identify the most frequent cipher symbol as "e," then use bigram/trigram patterns plus word-structure reasoning to reconstruct the rest of the alphabet. Kerckhoffs demonstrates this on a worked cryptogram, deducing "Il a été difficile de lire ses lettres" letter-group by letter-group, and closes by re-enciphering the same sentence under the §1.0.3 formatting rules to show how much harder it becomes once word breaks and doubled letters are removed. He also notes a rule of thumb: the longer a cryptogram, the easier it is to break — a single full line is usually enough.

Breaking polyalphabetic ("double-key") ciphers — Kerckhoffs' most technically original contribution. His procedure has four parts, each of which he works through with real examples (the worked detail below is expanded in Ed. 3 directly from the Part II scan):

1. **Determine key length**, via repeated letter-group analysis. Kerckhoffs states the principle generally: in any enciphered text, two identical polygrams are the product of two identical plaintext groups enciphered with the same alphabets, and the number of characters between two such repeats must be a multiple of the key length — so the key length is simply the common factor of the intervals between repeats. He illustrates this first on the phrase "Vous ne pouvez vous défendre sans vous exposer," then on a 75-letter sentence ("la présence de soldats ennemis nous a de nouveau été annoncée ce matin de différents côtés," intervals stripped), which yields 18 repetitions — 3 trigrams (nou, sen, nce) and 15 bigrams (en, es, de, ce, no, te, ts, etc.) — enough to pin the key length regardless of whether a 5-, 6-, 7-, 8-, 9-, or 10-letter key was used. He then applies the method to an actual cryptogram beginning "RMUUWQPMQGXHWBGGKKKNITMUXWWTMGGXHEPH," finding one trigram (GXH) and three bigrams (MU, GG, TM) whose intervals (21, 15, 6, 21 characters) share the common factor 3, correctly identifying a 3-letter key.

2. **Determine alphabet ordering — non-inverted alphabets.** Splitting the cryptogram above into 3-character slices and grouping by column position, Kerckhoffs shows that once the cipher digit standing for “e” is known in one column, the digits for every other letter in that same alphabet are known too, whether the cryptogram used the plain Vigenère table or the Saint-Cyr/Gronsfeld systems — because in a non-inverted alphabet the relative spacing between letters never changes. Working through the three columns of his example, he recovers the key cid (equivalently, 2-8-3 under Gronsfeld numbering) and decodes the message as “Personne ne peut déchiffrer votre dépêche” (“No one can decipher your dispatch”). For cases where the “e” column is ambiguous from frequency alone, he gives a tie-breaking heuristic: because K and W almost never appear together in French, any candidate alphabet that would require a high coefficient on the ciphertext digits standing for K and W can be rejected outright.
3. **Alphabets irregularly inverted — the Havas/Reuters telegram.** When straightforward frequency analysis on the “e” columns fails to yield a readable plaintext, Kerckhoffs shows how to attack the opening letters directly using word-structure and contextual reasoning, illustrated with a real intercepted 1882 Reuters/Havas dispatch from London about the British Egypt campaign. Working from just the first few recovered “e” positions, he reconstructs the message almost entirely by elimination and vocabulary guessing before confirming letters against frequency: ruling out an opening infinitive or present participle on stylistic grounds, then ruling out common nouns, certain adjectives, and most function words that don’t fit the known e positions, he narrows the opening word to le over ce (on frequency grounds), then to le général (the only word fitting a required e…e pattern), then — anticipating names like Arabi, Wolseley, Suez, Ismaïlia, canal, général, soldats — to Wolseley as the general’s name, then télégraphie as the verb, then qu’il attend seulement le service de transports et de communications, and finally soit complètement organisé pour faire une nouvelle marche en avant (“is only waiting for transport and communications service… to be completely organized to make a new advance”), at which point he stops, leaving the rest as an exercise for the reader.
4. **Symmetry of position** — Kerckhoffs’ own named technique, which he states is unpublished anywhere else: if the substitution alphabets are all shifts of one square table with a fixed (even if scrambled) letter order, as in Vigenère/Saint-Cyr/Gronsfeld, then once the mapping for one letter is known in two different alphabets of the key, the mapping for every other letter in the second alphabet can be derived by simple arithmetic from its known position in the first, because relative letter spacing is preserved across all shifted alphabets. He gives a fully worked numeric illustration (new in Ed. 3): given a 3-alphabet key where 19 digit values are already known (EPBLAFN for alphabet 1, UAT for alphabet 2, TDFHKMRSIC for alphabet 3), and noting that alphabets 1–2 share the plaintext letter A and alphabets 2–3 share T, every other known letter can be transplanted between alphabets by placing it at the same distance from the shared letter — extending the known coverage from 19 to 46 digit values without any further guessing. Applied retroactively to the Havas telegram above, he notes that using this trick from the start would have let him stop guessing after only the fifth word.
5. **Multiple ciphertexts under one key — 12 short cryptograms.** If several short messages share a key, stacking them lets the repeated-group technique be applied across messages instead of within one, easing key-length recovery even for very short texts. Kerckhoffs demonstrates this with a set of a dozen very short cryptograms (new in Ed. 3), all enciphered with the same phrase, “je me mets sur la défensive” (“I am putting myself on the defensive”), noting in passing that of the 22 letters making up that key, 3 repeat, leaving only 14 effectively distinct alphabets — a situation the decipherer must learn to exploit rather than be thrown by. Working through cryptograms IV, V, VI, and VII, he recovers, in turn, l’armée (from a mandatory -ée word ending), le général (reusing the earlier Havas-telegram reasoning), ferez/serez… vous (disambiguated only later), and ne renvoyez (where symmetry of position rules out an alternative reading), illustrating how quickly the pool of known letters compounds once even a few words are cracked across

several short messages.

Breaking the variable-key (“running”) system. A dispatch enciphered this way is only readable once the decipherer relocates the stop letter; this is hard on a very short message, but a dispatch of four or five lines shows a detectable regularity in the irregularity — the stop letter is, by construction, the one digit that never repeats twice in a row. Once found, the intervals between successive stop letters can be tabulated in columns and analyzed exactly as in the fixed-key case. Kerckhoffs notes that he was able to break every cryptogram of this type submitted to him by the Military Telegraphy Commission in under two hours, except where the alphabet itself had also been irregularly scrambled.

A triple-key cipher (new in Ed. 3). Having shown that no variable-base system in contemporary use offers serious guarantees of indecipherability, Kerckhoffs proposes — mostly as a curiosity, and admitting it is impractical for real battlefield use — combining the Saint-Cyr substitution system with the columnar transposition method from Part I under two independent keyword-derived keys: a short adjective supplying the substitution key and a longer noun supplying the transposition formula. He works a full example: enciphering “Vous ferez ce soir une attaque simulée” with substitution key sénat romain, where romain yields the numeric transposition formula 6-5-3-1-2-4, producing the final cryptogram “dcawibeumxkzrrkbimwyhnsswwzvrvudlame.” He reiterates that this scheme, however secure, is too impractical to recommend for real military service, and offers it “only as a curiosity” demonstrating that dispensing with secrecy entirely is achievable in principle.

Breaking coded dictionaries: shown via the worked numeric example in §1.2.C above — guessing the plaintext meaning of just two intercepted code groups and comparing assumed vs. actual numbers reveals the underlying permutation formula and additive key, after which the rest of the dictionary-coded message becomes readable.

1.5 Kerckhoffs’ Closing Thesis and Legacy

Kerckhoffs closed the article by generalizing his critique: transposition-only systems are secure only if the transposition mechanism itself is secret; multi-alphabet substitution systems are breakable if regular/systematic, and impractical if made complex enough to resist his own analytical methods; coded dictionaries are the most secure in practice but violate the “no secrecy required” desideratum outright. He proposed the triple-key system above as a curiosity, admitting it was impractical for real battlefield use.

He ended by endorsing a maxim from the 17th-century cryptographer Jean Robert du Carlet (from du Carlet’s 1644 treatise La Cryptographie) as the ultimate design standard for any military cipher: “Ars ipsi secreta magistro” — a cipher is only good if it remains undecipherable even to the master who invented it. This is the historical ancestor of the modern practice of subjecting cryptographic algorithms to public, adversarial scrutiny rather than relying on any one party’s — including the designer’s — confidence in its secrecy. Writing nearly a century later, Phillip Rogaway’s “The Moral Character of Cryptographic Work” (see §18.1) echoes the same insight in modern form: a cipher’s design must never be allowed to become an institutional point of vulnerability.

Note on the source text. This edition draws on direct scans of both parts of Kerckhoffs’ article. Part I (signed “Aug. Kerckhoffs, Docteur ès lettres, Professeur à l’École des hautes études commerciales et à l’École Arago,” marked “(À continuer.)”) covers §I “La cryptographie dans l’armée” (§1.0–1.0.3 above), §II “Desiderata de la cryptographie militaire” (§1.1), and the opening of §III “Les différentes méthodes de cryptographie” (transposition and substitution/interversion ciphers, §1.2.A–B). Part II (pp. 161–191), consulted directly for Editions 3–5 rather than only via the Petitcolas digest, covers the remainder of §III (cryptanalysis of double-key systems, the variable-key system, and the triple-key curiosity — §1.4 above), §III.C “Dictionnaires chiffrés” (§1.2.C above), §IV “Les Cryptographes” (§1.3

above), and the closing du Carlet maxim (§1.5 above). Part II is signed identically and carries the same footnote referencing Kasiski's Die Geheimschriften und die Dechiffrirkunst (1863) and Fleissner's Handbuch der Kryptographie.

Summary table: then vs. now

Kerckhoffs (1883)	Modern equivalent
Principle 2: system must not require secrecy of the method	Kerckhoffs' Principle — algorithms should be public; only keys are secret
"The enemy knows the system"	Shannon's Maxim
Rejection of secret dictionaries/mechanisms as a security basis	Rejection of "security through obscurity"
Symmetry-of-position technique for breaking Vigenère-family ciphers	Precursor to the Kasiski examination / modern polyalphabetic cryptanalysis
"Ars ipsi secreta magistro" (du Carlet, 1644)	Open, peer-reviewed cryptographic design (e.g., public algorithm competitions like AES/SHA-3)
Letter-frequency & repeated-group analysis	Index of Coincidence and classical cryptanalysis (see §14.1)
Coded-dictionary key recovery via known-plaintext digit comparison	Chosen/known-plaintext attacks on keyed codebooks (see §14.1)

2. Mathematical and Theoretical Foundations

2.1 Number Theory

Modular Arithmetic: foundation of public-key cryptography; operations mod n create finite rings/fields.

Prime Numbers: central to RSA and Diffie-Hellman; primality testing via Miller-Rabin, AKS.

Discrete Logarithm Problem (DLP): given g and g^x mod p, finding x is infeasible for large primes. Underpins Diffie-Hellman, ElGamal, ECC.

Integer Factorization: given n = p*q, finding p, q is hard. Underpins RSA.

Euclidean Algorithm: computes gcd(a,b). The Extended Euclidean Algorithm derives Bezout coefficients s, t satisfying sa + tb = gcd(a,b), enabling modular inverses a^-1 mod m.

Euler's Totient Function and Fermat/Euler Theorems (new in Ed. 5): Euler's totient (phi) function φ(n) counts the integers in [1, n] that are coprime to n; for a prime p, φ(p) = p − 1, and for an RSA modulus n = p·q with distinct primes, φ(n) = (p − 1)(q − 1). Fermat's Little Theorem states that for a prime p and any integer a not divisible by p, a^(p−1) ≡ 1 (mod p); Euler's Theorem generalizes this to any modulus n, stating that a^φ(n) ≡ 1 (mod n) whenever gcd(a, n) = 1. These two results are the number-theoretic basis for why RSA decryption correctly inverts encryption: choosing the private exponent d as the modular inverse of e modulo φ(n) guarantees $(m^e)^d \equiv m \pmod n$ via Euler's Theorem, and are likewise foundational to the correctness proofs of Diffie-Hellman-family schemes.

Modular Exponentiation: repeated squaring achieves O(log e) multiplications for x^e mod m.

2.2 Abstract Algebra

Groups: set with associative operation, identity, inverses. Cyclic groups of prime order underlie discrete-log cryptography.

Rings and Fields: finite fields F_p (prime fields) and F_(2^m) (binary extension fields) underlie AES and ECC respectively.

Galois Field Arithmetic F_(2^m): elements represented as polynomials A(x) = sum a_i x^i mod P(x). AES uses irreducible polynomial P(x) = x^8 + x^4 + x^3 + x + 1. Inversion uses the Extended Euclidean Algorithm on polynomial rings.

Elliptic Curves: y^2 = x^3 + ax + b mod p over F_p, with non-singularity condition 4a^3 + 27b^2 != 0 mod p. Points form an abelian group under geometric addition.

Lattice Theory: a lattice Lambda subset of R^n is generated by integer combinations of basis vectors B = {b_1,...,b_k}. Key concepts:

- Fundamental Parallelepiped: P(B) = {sum x_i b_i | 0 <= x_i < 1}
- Lattice Volume: det(Lambda) = sqrt(det(B^T B))
- Unimodular Transformation: basis equivalence via integer matrices U with det(U) = +/-1
- Successive Minima lambda_i(Lambda): smallest radius containing i linearly independent lattice vectors.
- Minkowski's Theorem: lambda_1(Lambda) <= sqrt(n)*det(Lambda)^(1/n).

2.3 Complexity Theory

One-Way Functions (OWFs): f:{0,1}->{0,1} computable in polynomial time, where no probabilistic polynomial-time (PPT) algorithm can find a preimage except with negligible probability. A OWF is termed a trapdoor one-way function/permutation if some auxiliary secret information (the "trapdoor") makes inversion easy for whoever holds it, while the function remains one-way to everyone else — the abstract property RSA and other public-key encryption/signature primitives are built on. OWFs also come as collections (families indexed by a public index, e.g., one function per RSA modulus) rather than as a single fixed function, which is the form most concrete number-theoretic candidates actually take.

Goldreich-Levin Theorem: for any OWF f, the inner product B_r(x) = <x,r> mod 2 is a hard-core predicate — unpredictable from (f(x), r) better than 1/2 probability.

Blum-Micali Construction: any OWF with a hard-core predicate can be iteratively evaluated to stretch a random seed into a cryptographically secure pseudorandom sequence.

NP-Complete Problems: used in interactive proofs; Graph Three-Colorability is the canonical zero-knowledge example (see §8.2).

Weak vs. Strong One-Way Functions (new in Ed. 4): a weak one-way function is only guaranteed to be hard to invert with some non-negligible (rather than overwhelming) probability; a foundational result of the field is that any weak one-way function can be transformed into a strong one (invertible only with negligible probability) by evaluating many independent copies in parallel and combining their outputs, which is why "one-way function exists" is normally treated as a single binary assumption rather than a spectrum.

2.4 Probability and Information Theory

Shannon's Perfect Secrecy: P[M=m | C=c] = P[M=m] — observing the ciphertext gives an adversary no information at all about the message. The One-Time Pad is the only perfectly secure cipher (key as long as message, used once, truly random).

Information-Theoretic vs. Computational Security: the former holds against unbounded adversaries; the latter relies on assumed problem hardness and bounded (polynomial-time) adversaries.

Entropy: measures uncertainty; high-entropy sources are essential for key generation. Low-entropy key derivation is a common real-world vulnerability (see §13.6).

2.5 Formal Security Definitions and the Provable-Security Paradigm

Modern (post-1980s) cryptography is distinguished from the classical, ad-hoc cipher design surveyed in §1 by its insistence on three disciplines: precise definitions of what "secure" means, explicit assumptions about what problems are hard, and mathematical proofs of security that reduce a scheme's security to those assumptions. This shift — from "does anyone see how to break it?" to "can we prove no efficient adversary can break it, assuming problem X is hard?" — is the central methodological difference between the Kerckhoffs-era ciphers of §1 and everything in §3 onward.

Security as a game, not a property: a scheme is defined secure relative to an explicit adversarial "experiment" (game) in which a probabilistic polynomial-time adversary interacts with the scheme and must win with only negligible advantage over guessing.

Semantic security: informally, ciphertexts leak no partial information about the plaintext to any efficient adversary, no matter what side information the adversary has — the encryption-scheme analogue of perfect secrecy, adapted to computationally bounded adversaries.

Indistinguishability (IND) games — the practical formulation of semantic security:

- **IND-CPA** (chosen-plaintext attack): the adversary may request encryptions of plaintexts of its choice, then must distinguish the encryption of one of two chosen plaintexts from the other, with negligible advantage over 1/2.
- **IND-CCA1** (lunchtime attack): the adversary additionally gets decryption-oracle access before seeing the challenge ciphertext.
- **IND-CCA2** (adaptive chosen-ciphertext attack): the adversary retains decryption-oracle access even after seeing the challenge ciphertext (barred only from querying the challenge itself) — the strongest standard notion, matching real-world attacks like padding-oracle attacks (§3.1, §14.3).

2.5.1 Computational Indistinguishability (new in Ed. 4)

A more general notion underlying IND-CPA/CCA, pseudorandomness (§2.5, building blocks below), and zero-knowledge simulation (§8.1) alike: two probability ensembles (families of distributions indexed by a security parameter) are computationally indistinguishable if no efficient (PPT) algorithm, given a sample from either, can guess which one it came from with more than negligible advantage over random guessing — even though the two ensembles may be statistically very different (even disjoint in support). This is a strictly computational relaxation of statistical closeness, and is the common technical currency that lets otherwise-different security definitions (semantic security, PRG/PRF security, zero-knowledge simulation) all be phrased as "distribution A is indistinguishable from distribution B."

Building blocks with formal definitions:

- **Pseudorandom Generator (PRG):** stretches a short truly-random seed into a longer string computationally indistinguishable from true randomness.
- **Pseudorandom Function (PRF):** a keyed function family indistinguishable, to any efficient adversary with oracle access, from a truly random function. The classical Goldreich–Goldwasser–Micali (GGM) construction builds a PRF out of any length-doubling PRG by using the PRG's two output halves to label the two children of each node in a binary tree of depth n, so that the key seed at the root determines a pseudorandom leaf label for every n-bit input — giving an efficiently-evaluable PRF from a strictly weaker primitive (a PRG) without needing any new hardness assumption.
- **Pseudorandom Permutation (PRP):** a PRF that is also efficiently invertible given the key — the formal model for a block cipher (§3.1). "Strong" PRP security also requires indistinguishability when the adversary can query the inverse direction.

Proof by reduction: the standard proof technique — assume a hypothetical adversary that breaks the scheme's security game, then show how to use that adversary as a subroutine to solve the underlying hard problem. Since the hard problem is assumed infeasible, no such

adversary can exist. The hybrid argument is the standard tool: a sequence of intermediate ("hybrid") distributions is constructed between the real and ideal experiments, and adjacent hybrids are shown indistinguishable one step at a time.

Message authentication and signatures: the analogous strong security notion is existential unforgeability under chosen-message attack (EU-CMA).

The KEM/DEM (hybrid encryption) paradigm: public-key operations are slow, so practical public-key encryption combines a Key Encapsulation Mechanism (uses the public key just to establish a fresh symmetric session key) with a Data Encapsulation Mechanism (a fast symmetric AEAD scheme, §3.3) — the structural basis of TLS (§7.1) and most real-world hybrid encryption.

Provable security vs. real-world security: a security proof only guarantees that breaking the scheme is at least as hard as breaking the underlying assumption within the model as defined — it does not cover implementation flaws, side channels (§13), protocol-composition errors, or misuse. This gap is a recurring theme throughout §13–14 and §18 below.

2.6 The Random Oracle Model (new in Edition 3)

Many efficient public-key schemes (RSA-OAEP, RSA-PSS, Fiat-Shamir signatures) cannot be proven secure against the standard security games of §2.5 within the "standard model" of computation — the reduction only goes through if the hash function used inside the scheme is treated as behaving like an idealized random function. Mihir Bellare and Phillip Rogaway formalized this idealization as the Random Oracle Model (ROM).

The oracle abstraction: a cryptographic hash function is modeled as a publicly accessible, perfectly random oracle. When any party — honest participant or adversary — queries the oracle on an input, the oracle checks its internal table: if that input has been queried before, it returns the same output as before; if the input is new, it samples a fresh, uniformly random output, records it, and returns it.

Observability: because no party can compute a "hash" value without going through the oracle, a security reduction working inside the ROM can see every query an adversary makes to it — letting the reduction monitor exactly what the adversary is probing.

Programmability: the reduction is also allowed to choose the oracle's output for any input the adversary has not yet queried, as long as the values it hands back remain uniformly distributed. This lets a reduction secretly embed an instance of a hard problem into the oracle's outputs, so that an adversary who breaks the scheme is forced, along the way, to solve that hard problem.

The Canetti–Goldreich–Halevi (CGH) result (1998): Ran Canetti, Oded Goldreich, and Shai Halevi proved that the ROM is formally unsound as an abstraction — they constructed an artificial digital signature scheme that is provably secure when the hash function is treated as a true random oracle, yet becomes completely insecure the moment it is instantiated with any concrete, deterministic hash function. This shows that a ROM security proof is a heuristic guarantee, not an unconditional one: it certifies that a scheme has no structural design flaw under idealized hashing, but says nothing about pathological interactions a real, deterministic hash function might have with the rest of the protocol.

Why the ROM remains standard practice anyway: despite the CGH counterexample being a deliberately constructed pathology rather than a real-world break, the ROM remains essential for instantiating efficient asymmetric primitives, including RSA-OAEP (§6.1, CCA2 security via multi-stage hash padding), RSA-PSS (§6.1, probabilistic signatures with randomized salts), and the Fiat-Shamir transformation (§8.4, converting interactive sigma protocols into non-interactive signatures). In practice, ROM proofs are treated as strong evidence of sound design rather than as absolute guarantees, with the residual gap between the model and concrete hash functions folded into the broader provable-security-vs-real-world-security caveat of §2.5.

3. Symmetric Cryptography

3.1 Block Ciphers

Confusion and Diffusion (new in Ed. 5): Claude Shannon's 1949 design principles for symmetric ciphers, still the conceptual basis of modern block-cipher design. Confusion means making the relationship between the key and the ciphertext as complex and non-linear as possible (typically supplied by substitution/S-box layers), so that statistical attacks like Kerckhoffs' own letter-frequency techniques (§1.4) gain no traction. Diffusion means spreading the influence of each plaintext bit (and each key bit) over many ciphertext bits (typically supplied by permutation/mixing layers), so that a single-bit change in the input flips roughly half the output bits (the "avalanche effect"). AES's SubBytes step supplies confusion and its ShiftRows/MixColumns steps supply diffusion; DES's S-boxes and P-box play the analogous roles within its Feistel structure.

AES (Advanced Encryption Standard) - Designers: Joan Daemen and Vincent Rijmen (Rijndael); Standard: NIST FIPS 197 (2001). - Block size 128 bits; keys 128/192/256 bits; rounds 10/12/14 respectively. - Structure: Substitution-Permutation Network (SPN); operations: SubBytes, ShiftRows, MixColumns, AddRoundKey. - Formally modeled as a keyed Pseudorandom Permutation (§2.5.1); no practical attacks on full AES exist — related-key attacks apply only to reduced-round variants. - Hardware acceleration: AES-NI (Intel/AMD), ARMv8 Crypto Extension.

DES / 3DES - DES: 64-bit block, 56-bit effective key, 16 rounds — broken by brute force (Deep Crack, 1998). - 3DES: applies DES three times; 112/168-bit keys; vulnerable to SWEET32 (64-bit block); deprecated by NIST.

Key Whitening and DESX (new in Ed. 5): Key whitening XORs additional key material into the plaintext before the first round and/or into the state after the last round of a block cipher, cheaply increasing effective key length and raising the cost of exhaustive-search and some analytical attacks without touching the cipher's internal rounds. DESX (Ron Rivest) is the canonical example: it whitens plain DES with two extra 64-bit keys (one XORed in before encryption, one after), raising the effective brute-force cost far above DES's native 56-bit key while reusing unmodified DES internals; AES's own AddRoundKey layers are a related, per-round form of the same idea.

Modes of Operation

Mode	Description	Properties	Security Notes
ECB	Independent block encryption	Parallelizable, deterministic	Insecure — reveals plaintext patterns
CBC	XORs previous ciphertext block	Sequential decryption	Needs unique IV; padding-oracle risk
OFB (new in Ed. 5)	Encrypts the IV repeatedly to build a keystream, independent of the plaintext/ciphertext, which is then XORed with the data	Parallelizable keystream generation (precomputable), self-synchronizing only via a fresh IV, no error propagation beyond the affected bit	Bit-flipping malleable like a stream cipher; a keystream reused across messages (e.g., via IV reuse) is catastrophic
CFB (new in Ed. 5)	Encrypts the previous ciphertext block (or IV) to build a keystream segment, then XORs it with the plaintext, feeding the new ciphertext back in	Turns a block cipher into a self-synchronizing stream cipher; sequential encryption but parallelizable decryption	Needs unique IV; same bit-flipping malleability as OFB/CTR
CTR	Encrypts counter as keystream	Parallelizable, random access	Secure with unique nonce

GCM	CTR + GHASH authentication	AEAD, parallelizable	Catastrophic failure on nonce reuse
CCM	Counter + CBC-MAC	AEAD, constrained environments	Slower than GCM, two passes
XTS	Tweaked codebook + ciphertext stealing	Disk encryption, per-sector tweak	No authentication

Other Block Ciphers: Blowfish/Twofish (Schneier; Twofish was an AES finalist), Serpent (AES finalist, conservative margins), Camellia (Japan), ARIA (South Korea).

Legacy 1990s Block Ciphers (new in Ed. 4). Several block ciphers from the pre-AES era remain historically significant reference designs, standardized or widely deployed in their day, though all have been superseded by AES for new designs:

- **IDEA** (International Data Encryption Algorithm): 64-bit block, 128-bit key, 8.5 rounds, mixing XOR, addition mod 2^16, and multiplication mod 2^16+1 from three different algebraic groups; used in early PGP versions.
- **RC5 / RC2** (Ron Rivest, RSA Security): RC5 is a parameterized cipher (variable block size, key size, and round count) built from data-dependent rotations; RC2 is a 64-bit-block cipher used historically in early SSL/S-MIME cipher suites.
- **FEAL** (Fast Data Encipherment Algorithm, NTT Japan): an early Feistel cipher designed for software speed; successive versions (FEAL-8, FEAL-N, FEAL-NX) were progressively broken by differential and linear cryptanalysis, making FEAL an important historical case study in both attack techniques (§14.2).
- **GOST 28147-89:** the Soviet/Russian government standard block cipher, a 64-bit-block, 256-bit-key Feistel cipher with a simple round function but a large number of rounds (32).
- **LOKI (’89/’91):** Australian-designed 64-bit-block Feistel ciphers; LOKI’91 was a revision addressing differential-cryptanalysis weaknesses found in LOKI’89.
- **SAFER** (Secure And Fast Encryption Routine): a family of byte-oriented, non-Feistel (SPN-like) ciphers designed by James Massey for Cylink Corporation.

3.2 Stream Ciphers

ChaCha20 / Salsa20 - Designer: Daniel J. Bernstein (djb); ARX structure (Add-Rotate-XOR) on 32-bit words. - Key 256 bits; nonce 96 bits (XChaCha20: 192 bits); 20 rounds. - No lookup tables -> inherently constant-time; fast without hardware acceleration. - RFC 8439; used in TLS 1.3, WireGuard, Signal.

Other Stream Ciphers: RC4 (broken, deprecated), A5/1 & A5/2 (GSM, both broken/weak), E0 (Bluetooth, vulnerable to correlation attacks), Grain/Trivium (eSTREAM; Trivium uses 288-bit state across 3 NLFSRs).

Linear Feedback Shift Registers (LFSRs): m-stage LFSRs governed by feedback polynomial P(x) = 1 + c_1 x + … + c_m x^m over F_2. Vulnerable to known-plaintext attacks via matrix solving (2m plaintexts recover the feedback polynomial).

3.3 Authenticated Encryption (AEAD)

Provides confidentiality + authenticity together (an AEAD scheme is IND-CCA2 secure by construction when built correctly, §2.5); associated data is authenticated but not encrypted.

Constructions: AES-GCM (widely deployed, careful nonce management required), AES-CCM (Wi-Fi WPA2/3, Bluetooth LE), ChaCha20-Poly1305 (preferred without AES-NI, constant-time by design), AES-GCM-SIV (synthetic IV, robust against nonce reuse).

ChaCha20-Poly1305 vs AES-256-GCM

Feature	ChaCha20-Poly1305	AES-256-GCM
Type	Stream cipher + MAC	Block cipher (CTR) + GHASH

Key size	256 bits	128/192/256 bits
Nonce	96 bits (XChaCha20: 192)	96 bits
Auth tag	128 bits (Poly1305)	128 bits (GHASH)
HW acceleration	None needed	AES-NI, ARMv8 CE
Constant-time	Yes by design	Implementation-dependent
FIPS 140-2	No	Yes
TLS 1.3 suite	TLS_CHACHA20_POLY1305_SHA256	TLS_AES_256_GCM_SHA384
Speed, no AES-NI	~4,200 MB/s	~1,800 MB/s
Speed, with AES-NI	~396-1,158 MB/s	~577-2,617 MB/s
Post-quantum margin	~128-bit	~128-bit

4. Cryptographic Hash Functions

4.1 Required Properties

1. Preimage resistance: given y, hard to find x with H(x)=y.
2. Second-preimage resistance: given x, hard to find x'!=x with H(x')=H(x).
3. Collision resistance: hard to find any x!=x' with H(x)=H(x').

4.2 Hash Function Families

SHA-2 (FIPS 180-4): SHA-256 (256-bit, most widely used), SHA-384/SHA-512 (SHA-512 gives 256-bit collision resistance), truncated variants (SHA-224, SHA-512/224, SHA-512/256). Merkle-Damgard construction with Davies-Meyer compression. ~1.2 GB/s single-core, sequential only.

SHA-3 / Keccak (FIPS 202): 2012 NIST competition winner. Sponge construction (not Merkle-Damgard). Variants: SHA3-224/256/384/512, SHAKE128/256 (XOFs). Resistant to length-extension attacks. ~0.8 GB/s, not parallelizable.

BLAKE Family: - BLAKE2 (2012): faster than MD5 at SHA-3-equivalent security; BLAKE2b (64-bit platforms), BLAKE2s (32-bit); built-in keyed hashing (no HMAC needed); used in Linux kernel RNG, WireGuard, Argon2. - BLAKE3 (2020, Aumasson/O'Connor/Neves/Wilcox-O'Hearn): Merkle tree over 1 KiB chunks + BLAKE2s-derived compression; parallel scaling (~1 GB/s single-core to 30+ GB/s on 32 cores); modes for hashing, MAC, and KDF; not yet FIPS.

Broken (historical): MD5 (128-bit, practical collisions since 2004), SHA-1 (160-bit, SHAttered collision 2017, chosen-prefix collisions demonstrated) — both deprecated.

4.3 Performance Comparison

Algorithm	Speed (GB/s)	Output	Security Level	Status
MD5	~0.7	128 bits	Broken	Do not use
SHA-1	~0.9	160 bits	Broken	Deprecated
SHA-256	~1.2	256 bits	128-bit	Standard
SHA-512	~1.8	512 bits	256-bit	Standard
SHA3-256	~0.8	256 bits	128-bit	Standard
BLAKE2b	~2.0	up to 512 bits	128-bit	Widely used
BLAKE3	~6.0+	256 bits (extendable)	128-bit	Modern choice

4.4 Hash-Based Constructions

HMAC: HMAC(K,m) = H((K'⊕opad) || H((K'⊕ipad) || m)); provably secure under reasonable hash assumptions (FIPS 198-1, RFC 2104).

HKDF: Extract-then-Expand key derivation (RFC 5869); used in TLS 1.3, Signal.

Merkle Trees: binary tree of hashes for efficient verification of large datasets; used in blockchain, certificate transparency, file integrity.

5. Message Authentication Codes (MACs)

HMAC: uses a hash function (typically SHA-256); most widely deployed MAC; the practical instantiation of the EU-CMA notion described in §2.5.

CMAC: uses a block cipher (typically AES); NIST SP 800-38B; no hash function needed.

GMAC: authentication component of AES-GCM; uses GHASH over GF(2^128); requires unique nonce.

Poly1305: one-time MAC by Bernstein; extremely fast in software; paired with ChaCha20 (TLS 1.3, WireGuard); key must be unique per message.

KMAC: based on SHA-3/Keccak's cSHAKE (NIST SP 800-185); variable output length.

6. Public Key Cryptography

6.1 RSA

Foundation: integer factorization.

Key generation: choose primes p,q -> n=pq, phi(n)=(p-1)(q-1) -> choose e (typically 65537) -> d = e^-1 mod phi(n) -> public (n,e), private (n,d). (Correctness of decryption follows from Euler's Theorem, §2.1.)

Operations: Encrypt c=m^e mod n; Decrypt m=c^d mod n; Sign s=m^d mod n; Verify m=s^e mod n.

Security notes: - Minimum key size 2048 bits (3072+ for long-term security). - Raw ("plain") RSA is deterministic and not IND-CPA secure (§2.5) — padding is essential. - RSA-OAEP: CCA2-secure encryption via multi-stage hash padding (Random Oracle Model, §2.6). - RSA-PSS: probabilistic signatures with randomized salts (Random Oracle Model, §2.6). - Bleichenbacher attacks: padding-oracle attacks on PKCS#1 v1.5. - ROCA (2017): weak randomness in Infineon TPM RSA key generation. - Common Modulus Attack: never share n between key pairs. - Small private exponent attacks: Wiener's and Boneh-Durfee attacks when d < n^0.292.

6.2 Diffie-Hellman Key Exchange and the ElGamal Cryptosystem

Diffie-Hellman Key Exchange. Foundation: Discrete Logarithm Problem. Protocol: agree on p, g -> Alice sends A=g^a mod p -> Bob sends B=g^b mod p -> shared secret s = B^a = A^b = g^(ab) mod p.

Security notes: Logjam (2015, downgrade to 512-bit export DH), FREAK (2015, similar RSA export downgrade), safe-prime requirement p=2q+1, minimum 2048-bit primes (3072+ recommended), ECDH as the more efficient elliptic-curve variant.

The ElGamal Encryption Scheme (new in Ed. 5). Taher ElGamal's 1985 public-key encryption scheme turns a Diffie-Hellman key exchange into full public-key encryption: the recipient's public key is (p, g, y=g^x mod p) for a secret x; to encrypt m, the sender picks a fresh random k, computes the ephemeral value c1 = g^k mod p and the masked message c2 = m · y^k mod p, and sends the pair (c1, c2); the recipient recovers m = c2 · $(c1^{x)}$-1 mod p. Because a fresh k is required for every encryption (reusing k leaks the relationship between

two plaintexts, similar to keystream reuse in a stream cipher), plain ElGamal is randomized rather than deterministic, giving it an inherent ciphertext-expansion cost (roughly double the plaintext size) relative to RSA. Plain ("textbook") ElGamal is only IND-CPA secure, not IND-CCA2 secure, without additional hardening; its security reduces to the Decisional Diffie-Hellman (DDH) assumption. ElGamal-family encryption is the conceptual basis of ECIES (the elliptic-curve integrated encryption scheme) and of the ElGamal-based threshold and homomorphic variants used in e-voting and mix-net systems.

6.3 Elliptic Curve Cryptography (ECC)

Advantage: RSA-equivalent security at much smaller key sizes.

Curve types: Weierstrass ($y^{2=x}3$+ax+b mod p), Montgomery (B$y^{2=x}$3+Ax^2+x, fast scalar multiplication), Edwards (x^{2+y}2=1+dx^{2y}2, complete addition formulas).

Standard curves: NIST P-256/P-384/P-521 (widely used, criticized for opaque parameter generation), Curve25519/X25519 (djb, 128-bit security, fast/constant-time), Curve448/X448 (224-bit margin), Ed25519 (signatures), secp256k1 (Bitcoin, Koblitz curve), BLS12-381 and BN254 (pairing-friendly, used in ZK-SNARKs).

Point addition (P=(x1,y1), Q=(x2,y2), P!=Q): - slope m = (y2-y1)/(x2-x1) mod p (doubling: m = (3x1^2+a)/(2y1) mod p) - x3 = m^2 - x1 - x2 mod p - y3 = m(x1-x3) - y1 mod p

6.4 Digital Signature Algorithms

The ElGamal Signature Scheme (new in Ed. 5). The historical predecessor of DSA, also due to Taher ElGamal (1985), and structurally distinct from ElGamal encryption above even though both rely on the same discrete-log setup (p, g, y=g^x mod p). To sign a message m, the signer picks a fresh random k coprime to p−1, computes r = g^k mod p, then solves for s in the congruence m ≡ x·r + k·s (mod p−1), producing signature (r, s); a verifier accepts iff g^m ≡ y^r · r^s (mod p). As with the encryption scheme, reusing k across two signatures — or using a predictable/low-entropy k — leaks the private key x via simple linear algebra, the same underlying failure mode later seen in real-world ECDSA nonce-reuse incidents (§6.4, "Critical vulnerability" note below). DSA (below) is essentially a variant of the ElGamal signature scheme redesigned to produce a shorter signature and to work in a prime-order subgroup rather than the full multiplicative group mod p.

ECDSA - Domain: prime p, curve coefficients a,b, generator G, subgroup order n, cofactor h. - Sign: pick random k in [1,n-1]; (x1,y1)=kG; r=x1 mod n; s=k^-1(z+rd) mod n; signature=(r,s). - Verify: u1=zs^-1 mod n, u2=rs^-1 mod n; P=u1G+u2H_A; valid iff x_P mod n == r. - Critical vulnerability: reusing nonce k across two messages under the same key allows private-key extraction via linear algebra. - Malleability: (r,s) can be rewritten as (r, -s mod L) without the private key — requires strict normalization.

EdDSA (Ed25519/Ed448) — djb et al. Deterministic nonce k=H(private_key || message) removes reliance on external entropy; fast constant-time; no malleability; compact 64-byte signatures. Ed25519: 128-bit security, ~30,000x faster verification than RSA-2048. Ed448: 224-bit margin.

DSA: predecessor to ECDSA using finite-field discrete logs, structurally derived from the ElGamal signature scheme above; deprecated in favor of ECDSA/EdDSA.

6.5 Key Encapsulation Mechanisms (KEMs)

Classical: RSA-KEM (no forward secrecy), ECDH-based KEMs (forward secrecy).

Post-quantum: ML-KEM/CRYSTALS-Kyber (FIPS 203, lattice-based, 3 parameter sets), HQC (NIST-selected additional code-based KEM, March 2025).

See §2.5 for the general KEM/DEM hybrid-encryption paradigm these are built for.

7. Key Exchange and Network Protocols

7.1 TLS

TLS 1.2 (RFC 5246, 2008) — 2-RTT handshake: Client Hello -> Server Hello/certificate -> key exchange (pre-master secret encrypted with server's public key) -> both derive session keys -> Change Cipher Spec + Finished. Issues: 100+ possible (often insecure) cipher suites, optional forward secrecy, downgrade-vulnerable version negotiation, compression enabled (CRIME/BREACH).

TLS 1.3 (RFC 8446, 2018) — 1-RTT handshake (0-RTT resumption with replay risk); AEAD-only cipher suites; mandatory forward secrecy (ephemeral DH); removed RSA key exchange, static DH, MD5, SHA-1, RC4, 3DES, compression; modern signatures (ECDSA, RSA-PSS; DSA removed); more of the handshake encrypted; explicit downgrade protection in ServerHello.random.

Cipher suites: TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256, TLS_AES_128_GCM_SHA256, TLS_AES_128_CCM_SHA256, TLS_AES_128_CCM_8_SHA256. As of January 2024, NIST requires TLS 1.3 support.

7.2 IPsec

IKEv2: IKE_SA_INIT (algorithm agreement, DH shared secret) -> IKE_AUTH (identity validation via certificates or PSKs).

Encapsulation: AH (integrity/origin authentication, no confidentiality) vs. ESP (confidentiality + integrity + anti-replay).

Key management: OAKLEY standardizes key establishment across certificates, PSKs, and third-party authorities.

7.3 Signal Protocol / Double Ratchet

X3DH: initial key agreement combining multiple DH operations.

Double Ratchet: Symmetric ratchet (KDF chain of message keys) + DH ratchet (asymmetric update each reply -> future secrecy).

Properties: forward secrecy, future secrecy (self-healing), deniability.

Used in Signal, WhatsApp, Facebook Messenger.

7.4 Noise Protocol Framework

Framework (not a single protocol) for building crypto protocols via named handshake patterns (e.g., Noise_XX, Noise_IK) describing initiator/responder static/ephemeral key usage. Used in WireGuard, Lightning Network, WhatsApp. Minimal, formally verified, no legacy baggage.

7.5 WireGuard

Modern VPN using Noise_IK. Crypto stack: Curve25519 (ECDH), ChaCha20-Poly1305 (AEAD), BLAKE2s (hashing), SipHash24 (hashtable keys). ~4,000 lines of code vs. 400,000+ for IPsec/OpenVPN; kernel-space; fast roaming.

7.6 Symmetric-Key Key Establishment and the Needham–Schroeder Protocol (new in Ed. 5)

Before public-key-based establishment (§6, §7.1–7.5) became practical, symmetric-key key establishment relied on a trusted third party — a Key Distribution Center (KDC) sharing a long-term key with every participant — to broker fresh session keys. The Needham–Schroeder symmetric-key protocol (1978) is the foundational scheme in this family: Alice contacts the

KDC naming herself and Bob; the KDC generates a fresh session key and returns it to Alice twice-encrypted — once for Alice directly, and once as a “ticket” pre-encrypted for Bob that Alice simply forwards to him — after which Alice and Bob exchange nonce-based challenge/response messages under the new session key to confirm mutual freshness. The protocol’s historically important weakness is that it does not let Bob verify the recency of the ticket he receives: if a session key is ever compromised, an attacker who recorded an old ticket can replay it to impersonate Alice to Bob indefinitely, since Bob has no way to distinguish a stale ticket from a fresh one (the flaw that later motivated Kerberos’s addition of explicit timestamps, and that Kerckhoffs’ own Principle 3 — keys must be easy to change — anticipates in spirit, see §1.1). This class of KDC-based protocol underlies Kerberos and remains conceptually distinct from, but historically prior to, the asymmetric (public-key) key-establishment protocols of §7.1–7.5.

8. Zero-Knowledge Proof Systems

8.1 Foundational Concepts

Introduced by Goldwasser, Micali, and Rackoff (1980s): a Prover (P) convinces a Verifier (V) that a statement is true, revealing nothing beyond that single bit of validity.

Three required properties: 1. Completeness — an honest Prover convinces an honest Verifier with probability 1. 2. Soundness — no cheating Prover can convince an honest Verifier of a false statement except with negligible probability. 3. Zero-knowledgeness — formalized via a Simulator (Sim) that, without the witness, produces a transcript indistinguishable from a real interaction (using the computational-indistinguishability notion of §2.5.1; when the simulated and real transcripts are only required to be computationally, rather than perfectly or statistically, close, the proof system is called computational zero-knowledge, as opposed to perfect or statistical zero-knowledge where the simulator’s output must match the real distribution exactly or near-exactly).

8.2 Interactive Proof Example: Graph Three-Colorability

(Formalized by Goldreich, Micali, and Wigderson.) Prover claims a valid 3-coloring of G=(V,E): 1. Prover randomly permutes colors, commits to each vertex’s color. 2. Verifier randomly picks an edge e=(u,v) in E. 3. Prover reveals the commitments for u and v. 4. Verifier checks the colors are distinct.

Cheating probability per round <= 1 - 1/|E|; after k rounds, Prob_cheat <= (1 - 1/|E|)^k. E.g., |E|=1000 with k=5000 rounds drives soundness error to negligible levels.

Proof vs. Proof of Knowledge: a Proof asserts a statement is true; a Proof of Knowledge asserts the Prover actually possesses a specific witness.

8.3 Schnorr Identification Protocol

Over cyclic group G, prime order q, generator g: 1. Prover picks random r in Z_q, sends r’=g^r mod p. 2. Verifier sends random challenge c in Z_q. 3. Prover computes s=r+ca mod q, sends s. Verifier checks g^s = r’*y^c mod p.

Knowledge Extractor: rewinds the Prover to obtain two responses s1,s2 for two challenges c1,c2 on the same commitment: s1=r+c1a, s2=r+c2a (mod q) implies a=(s1-s2)(c1-c2)^-1 mod q. This proves any Prover answering two distinct challenges must know the secret.

8.4 Non-Interactive ZK

Fiat-Shamir Transformation: converts interactive sigma protocols into non-interactive signatures by replacing the Verifier’s challenge with a hash of the commitment and message (relies on the Random Oracle Model, §2.6, for its security proof).

8.4a Witness Indistinguishability, Proofs vs. Arguments (new in Ed. 4)

Two related relaxations of zero-knowledge are useful when full simulation-based ZK is more than a protocol needs:

Witness Indistinguishability (WI): a weaker (and easier-to-achieve, and easier-to-compose) requirement than zero-knowledge — it only demands that, when a statement has multiple valid witnesses, the verifier cannot tell which witness the Prover used, even though the transcript may otherwise leak information about the statement itself. WI protocols compose in parallel more robustly than many zero-knowledge protocols do, which is useful in applications like certain identification schemes and ring-signature-style "prove one-of-many" constructions.

Proofs vs. Arguments (computational soundness): a "proof" system's soundness must hold against an unbounded cheating Prover; an argument system only requires soundness against a computationally-bounded (PPT) cheating Prover. Relaxing soundness to computational arguments is what allows arguments to be far more succinct than information-theoretic proofs — the property SNARKs and STARKs (§8.5) actually rely on, since a computationally unbounded prover could always find a satisfying witness or forge a short proof by brute force.

8.5 Modern ZK Proof System Comparison

Parameter	ZK-SNARKs	ZK-STARKs	Bulletproofs
Setup	Trusted Setup (SRS) via MPC ceremony	Fully transparent	Fully transparent
Primitives	Bilinear pairings (BN254, BLS12-381), KZG commitments	FRI + hash functions	Inner Product Args, Vector Pedersen Commitments
Prover complexity	O(N log N)	O(N*polylog N)	O(N log N)
Verifier complexity	O(1)	O(polylog N)	O(N) linear
Proof size	~200-500 bytes	~10-100 KB	~1-2 KB
Post-quantum	Vulnerable (Shor's)	Quantum-resistant (hash-based)	Vulnerable (Shor's)
Used in	Zcash, ZK-Rollups, Dark Forest	StarkNet, StarkEx	Monero, Grin, Beam

ZK-SNARKs: KZG commitments compress arithmetic circuits into single elliptic-curve elements; SRS from MPC ceremonies (e.g., Powers of Tau) — if the "toxic waste" leaks, false proofs become forgeable. Recursive SNARKs enable Incrementally Verifiable Computation (IVC) / Nova-style accumulation.

ZK-STARKs: FRI tests whether a committed evaluation vector matches a low-degree polynomial via split-and-fold: $f(x)=g(x^{2)+xh(x}2)$, folded via verifier challenge alpha, halving degree each step. No trusted setup, post-quantum resistant, larger proofs.

Bulletproofs: Inner product arguments prove committed values lie in a numeric range without revealing them; no trusted setup, small proofs, but linear verification — suited to range checks (e.g., confidential transactions) rather than large general circuits.

8.6 Circuit Development Stacks

Circom (compiles to R1CS, WASM provers, smart-contract verifiers), Halo2 (PlonKish arithmetization, custom gates/lookup tables, recursive composition without per-circuit trusted setup), Noir and ZoKrates (higher-level DSLs for verifiable privacy-preserving programs).

8.7 Arithmetization Techniques

R1CS: (Lz) o (Rz) = (O*z), where z=(1,x,w) is the witness vector and o is the Hadamard product.

AIR / PlonKish: converts execution traces into polynomial constraints with custom gates and permutation arguments.

Polynomial Commitment Schemes: KZG (pairing e:G1xG2->G_T, SRS ([1]1,[tau]1,...,[tau]^d 1); evaluate via quotient q(x)=(f(x)-v)/(x-a), proof pi=[q(tau)]1, single pairing check); IPA/Bulletproofs (pairing-free, recursive halo-style folding, O(log n) proofs, no trusted setup).

8.8 Advanced Interactive Proofs

Multivariate Sum-Check Protocol: verifies H = sum...sum P(x1,...,xv) for boolean inputs; Prover sends univariate polynomials gi(Xi) over v rounds; reduces O(2^v) evaluations to one.

GKR Protocol: interactive verification of layered arithmetic circuits, sublinear in circuit size (O(dv log S), d=depth, v=input vars, S=circuit width).

8.9 Algebraic Zero-Knowledge Identification Protocols (new in Ed. 4)

Beyond the discrete-log-based Schnorr protocol (§8.3), several other concrete zero-knowledge/witness-hiding identification protocols were developed in the 1980s–90s and remain standard reference constructions, each reducing to a different hard problem:

Feige–Fiat–Shamir (FFS) identification: based on the hardness of computing modular square roots (equivalently, the intractability of factoring); a prover holding a secret vector of modular square roots proves knowledge of it over several rounds of commit-challenge-response, with cheating probability halved per round (generalizing the basic Fiat-Shamir "Guess a square" scheme).

Guillou–Quisquater (GQ) identification: based on the RSA problem; more round-efficient than Fiat-Shamir-style schemes because each round achieves a much smaller soundness error, at the cost of RSA-sized (rather than modular-square-root-sized) computation per round. A signature variant (GQ signature) is obtained via the Fiat-Shamir transform (§8.4).

Ohta–Okamoto identification: a further discrete-log/RSA-hybrid variant offering different trust and efficiency trade-offs, and part of the same family of "sigma protocol" constructions as Schnorr, FFS, and GQ.

All three, like Schnorr, can be converted into non-interactive signature schemes via the Fiat-Shamir transformation (§8.4) and are the historical basis for many of the identification and signature standards referenced in §17.

9. Post-Quantum Cryptography

9.1 The Quantum Threat

Shor's Algorithm: uses Quantum Fourier Transform to find hidden subgroup periods in O((log N)^3) time — breaks factorization and discrete-log systems (RSA, ECC, DH, DSA).

Grover's Algorithm: quadratic speedup on unstructured search, roughly halving symmetric security levels (AES-256 -> 128-bit post-quantum security).

Harvest Now, Decrypt Later: adversaries collect ciphertext today to decrypt once quantum computers arrive.

Timeline: Global Risk Institute's 2026 estimate: a cryptographically relevant quantum computer is "quite possible" within 10 years, "likely" within 15.

9.2 NIST Post-Quantum Standards

Finalized (2024-2025): FIPS 203 ML-KEM/CRYSTALS-Kyber (lattice KEM, 3 parameter sets), FIPS 204 ML-DSA/CRYSTALS-Dilithium (lattice signatures), FIPS 205 SLH-DSA (stateless hash-based signatures), HQC (March 2025, code-based KEM).

Third-round signature candidates (2026): retained multivariate candidates UOV, MAYO, QR-UOV, SNOVA; CROSS and LESS eliminated on security concerns; updated submissions due Aug 14, 2026; standardization conference planned 2027.

9.3 Lattice-Based Cryptography

Hard problems: SVP (shortest nonzero vector), SIVP (n shortest linearly independent vectors), CVP/BDD (closest lattice vector to a target).

Learning With Errors (LWE): given A in Z_q^(mxn), b=As+e mod q (secret s, discrete-Gaussian error e), distinguish b from uniform random. Worst-case to average-case reduction: solving average-case LWE is as hard as worst-case GapSVP on arbitrary lattices.

Ring-LWE: structured variant over R_q=Z_q[X]/(X^n+1) for efficiency.

Module-LWE/Module-SIS: rank-d module formulation balancing security and key size — algebraic basis of ML-KEM and ML-DSA.

LLL Algorithm: polynomial-time basis reduction producing short vectors bounded by 2^((n-1)/2)*lambda1(Lambda).

Lattice Trapdoors / Gaussian Sampling: constructing A with trapdoor T to efficiently find short x with Ax=u mod q — blueprint for IBE and post-quantum signatures.

9.4 Code-Based Cryptography

McEliece Cryptosystem: hardness of decoding random linear codes; uses Goppa codes with hidden structure; very large public keys, fast operations. HQC is NIST’s selected code-based KEM.

9.5 Hash-Based Signatures

Lamport-Diffie one-time signatures: each signature reveals half the private key; secure if the hash is collision-resistant.

Merkle Signature Scheme: combines many one-time signatures in a tree. XMSS and LMS are stateful; SLH-DSA (SPHINCS+) is stateless and NIST-standardized.

9.6 Multivariate Cryptography

Based on hardness of solving multivariate quadratic equation systems over finite fields. Candidates: UOV, MAYO, QR-UOV, SNOVA. Advantages: small signatures, small public keys; but recent cryptanalysis has affected several parameter sets.

9.7 Isogeny-Based Cryptography

SIDH (supersingular isogeny DH): very small keys, but broken in 2022 by the Castryck-Decru attack (dimension-2 isogenies). CSIDH and other isogeny schemes remain under study.

9.8 Quantum Key Distribution (QKD)

BB84 (Bennett & Brassard, 1984): encodes bits in photon polarization across two bases; sifting discards mismatched-basis bits; eavesdropping detected via error-rate estimation; final key via error correction + privacy amplification. Security rests on the Heisenberg uncertainty principle and no-cloning theorem.

E91: uses quantum entanglement (EPR pairs); security via Bell inequality violation.

Decoy-state QKD: counters photon-number-splitting attacks using signal + decoy intensity states.

Integration with PQC: QKD gives information-theoretic key distribution; PQC handles authentication/signatures.

Market/deployment: QKD market projected at USD 13.66B by 2033 (23.2% CAGR); Toshiba achieved 254 km QKD over commercial fiber (April 2025); ID Quantique leads European deployment.

10. Fully Homomorphic Encryption (FHE)

10.1 Concept

Enables computation directly on encrypted data — f applied to ciphertexts yields an encryption of f(plaintext).

10.2 Scheme Taxonomy

BGV / BFV: exact modular integer arithmetic over polynomial rings Z_p; modulus switching and relinearization manage key-dimension growth; BFV is a simpler BGV variant.

CKKS: approximate fixed-point complex arithmetic; homomorphic rescaling manages precision; enables privacy-preserving ML inference.

TFHE: operates over the torus T=R/Z (or discrete T_N); bit-level Boolean ops (AND/XOR/MUX) as linear ciphertext combinations; fast programmable bootstrapping for arbitrary gates.

10.3 Bootstrapping

Homomorphically evaluates the decryption circuit using a public bootstrapping key to refresh a noisy ciphertext into a fresh one at baseline noise — enabling unlimited-depth computation.

10.4 Noise Growth

Addition adds small noise; multiplication adds significant (sometimes quadratic) noise. Managed via modulus switching and bootstrapping.

10.5 Applications (2026)

Healthcare (cross-institution drug discovery on encrypted data), financial services (credit scoring/fraud detection without sharing raw data), cloud computing (FHE-as-a-service across jurisdictions), privacy-preserving AI (FHE + MPC + ZKPs combined).

10.6 Libraries/Standardization

Microsoft SEAL (BFV, CKKS, BGV), HElib, PALISADE, Lattigo; NIST and the Homomorphic Encryption Standardization Consortium developing common standards.

10.7 Truth-Table Neural Networks (TT-TFHE)

Converts non-linear activations (ReLU, Sigmoid) into multi-input truth tables evaluated via TFHE lookup tables (e.g., Concrete-Numpy), enabling encrypted inference on datasets like MNIST/CIFAR-10.

11. Secure Multi-Party Computation (MPC)

11.1 Definition

Multiple parties jointly compute a function of their private inputs without revealing those inputs to one another; each party may receive a private output.

11.2 Security Models

Semi-honest (passive): adversaries follow the protocol but try to learn extra information.

Malicious (active): adversaries may deviate arbitrarily.

11.3 Fundamental Protocols

Yao's Garbled Circuits: one party garbles a Boolean circuit; the other evaluates it using Oblivious Transfer for their inputs; secure against semi-honest adversaries.

Oblivious Transfer (OT): 1-out-of-2 OT lets a Receiver learn exactly one of the Sender's two messages without revealing which, and without the Sender learning the choice. Foundational to many MPC protocols.

GMW Protocol: secret-shared computation over XOR/AND gates; more communication than garbled circuits, better for arithmetic circuits.

BGW / BMR: information-theoretic MPC for an honest majority.

11.4 Secret Sharing

Shamir's Secret Sharing: splits secret s into n shares via polynomial interpolation; any t shares reconstruct, t-1 reveal nothing (Lagrange interpolation over finite fields).

Additive Secret Sharing: s = x1+x2+…+xn mod q; all shares required to reconstruct.

Blakley's Secret Sharing (new in Ed. 4): an alternative, geometric threshold scheme published independently the same year as Shamir's (1979): each of n shares is an (n-1)-dimensional hyperplane in an n-dimensional space over a finite field, all constructed to pass through a common point representing the secret; any t of the n hyperplanes intersect at that single point, reconstructing the secret, while fewer than t hyperplanes leave the intersection under-determined. It is less space-efficient than Shamir's polynomial scheme (each share is larger relative to the secret) but is a historically important and conceptually distinct construction, illustrating that threshold secret sharing can be built from multiple different algebraic structures.

11.5 Threshold Cryptography

NIST Threshold Call (NISTIR 8214C): solicits threshold scheme proposals across Class N (standardized primitives) and Class S (special primitives) — Signatures (N1/S1), PKE (N2/S2), Symmetric (N3/S3), KeyGen (N4/S4), FHE (S5), ZKPoK (S6), Gadgets (S7). MPTS 2026 hosted 25 preview talks.

Threshold signatures: multiple parties jointly sign; no single party holds the full private key. Used in cryptocurrency custody and distributed key management.

11.6 The Ideal/Real Simulation Paradigm for Defining MPC Security (new in Ed. 5)

Beyond the specific protocols above (Yao, GMW, BGW/BMR), general secure multiparty computation is formally defined the same way zero-knowledge is (§8.1): by comparison to an idealized "ideal-world" execution in which every party privately sends its input to a fully trusted third party, who computes the function and privately returns each party's output, so that no party learns anything beyond its own output. A real-world protocol is then defined to securely realize a function if, for every real-world adversary attacking it, there exists an "ideal-world" adversary (a simulator) achieving an indistinguishable effect against the trusted-party version — meaning the real protocol leaks no more than the ideal, trusted-party computation

would. This is the same definitional move — replace “list every bad thing that shouldn’t happen” with “require indistinguishability from a provably safe idealized process” — that underlies semantic security (§2.5) and zero-knowledge simulation (§8.1), and it is what lets Yao’s garbled circuits, GMW, and BGW/BMR all be judged against one common yardstick despite their very different constructions.

11.7 Recent Research (2026)

Delayed Consistency Check Vulnerability: TPMPC 2026 research found that MPC protocols (Trident, Fantastic Four, SWIFT, Quad) which delay consistency checks allow malicious parties to extract information about intermediate wire values. Fix: replace delayed individual checks with a single joint consistency check.

12. Advanced Cryptographic Primitives

Identity-Based Encryption (IBE): Shamir (1984) proposed it; Boneh-Franklin (2001) gave the first practical pairing-based construction. Public key can be any string (e.g., an email address); requires a trusted Private Key Generator.

Attribute-Based Encryption (ABE): Ciphertext-Policy ABE (policy on ciphertext, attributes on keys) vs. Key-Policy ABE (policy on keys, attributes on ciphertext) — fine-grained access control.

Searchable Encryption: Symmetric Searchable Encryption (SSE) trades security/leakage for search efficiency; Deterministic Encryption (equality checks, leaks equality patterns); Order-Preserving Encryption (OPE, enables range queries, leaks approximate values).

Format-Preserving Encryption (FPE): ciphertext keeps the plaintext’s format (e.g., a credit-card-shaped output); NIST standards FF1/FF3-1 (SP 800-38G), based on Feistel networks.

Ring Signatures: signer signs anonymously on behalf of a “ring” of possible signers, unlinkable; used in Monero.

Group Signatures: a group member signs anonymously on the group’s behalf; a group manager can revoke anonymity.

Blind Signatures: David Chaum (1982); signer signs a message without seeing its content — digital cash, anonymous credentials, e-voting.

Anonymous Credentials: prove attribute possession without revealing identity; Direct Anonymous Attestation (DAA) used in TPMs.

Predicate / Functional Encryption: decryption succeeds only if a hidden predicate holds (predicate encryption); a key sk_f computes f(m) from Enc(m) without revealing m (functional encryption, generalizes IBE/ABE); Inner Product Encryption is a special case computing inner products.

Broadcast Encryption & Traitor Tracing: encrypt for a user subset with efficient revocation; traitor tracing identifies users who leaked keys.

Deniable Encryption: sender can plausibly deny a message’s content; alternate keys yield alternate plausible plaintexts — protects against coercion.

Verifiable Delay Functions (VDFs): require a fixed number of sequential steps to compute but are efficiently verifiable; used in randomness beacons, blockchain consensus, timestamping. Leading constructions: Wesolowski, Pietrzak.

Randomness Beacons: drand — distributed, publicly verifiable, unbiased randomness via threshold BLS signatures; used in blockchain, lotteries, protocols.

12.1 Additional Digital Signature Variants (new in Ed. 4)

Beyond the standard signature schemes of §6.4 and §9.5, several specialized signature variants address situations where an ordinary, universally-verifiable signature is either too strong or too weak a guarantee:

Undeniable Signatures (Chaum–van Antwerpen): verification requires the signer's cooperation via an interactive protocol — a valid signature cannot be checked by just anyone using a public key, giving the signer control over who can confirm authenticity. A companion disavowal protocol lets the signer interactively prove that a given signature is not theirs, preventing a signer from simply refusing to participate in verification to dodge a genuine signature.

Arbitrated Signatures: rely on a mutually trusted third party (an "arbiter") who mediates every signing and/or verification step, historically used in settings without strong enough public-key infrastructure to support direct verification.

Designated-Confirmer Signatures: a middle ground between ordinary and undeniable signatures — the signer designates a third party ("confirmer") who can also run the confirmation/verification protocol on the signer's behalf, so verification doesn't strictly require the original signer to be available.

Fail-Stop Signatures: provide a security guarantee against computationally unbounded forgers (rather than merely polynomial-time ones): if an unbounded adversary ever does forge a signature, the legitimate signer is guaranteed to be able to detect and cryptographically prove the forgery occurred, "stopping" further trust in the scheme rather than silently failing.

Password Hashing: must be slow, one-way, GPU/ASIC-resistant, and memory-hard. Argon2 (2015 Password Hashing Competition winner): Argon2d (GPU-resistant, side-channel-weak), Argon2i (side-channel-safe, weaker vs. GPUs), Argon2id (hybrid, recommended). bcrypt (Blowfish-based, adaptive cost, GPU-vulnerable), scrypt (memory-hard, used in Litecoin), PBKDF2 (NIST SP 800-132, iterative but not memory-hard, GPU/ASIC-vulnerable). Recommendation: use Argon2id for new systems.

Lightweight Cryptography: for constrained devices (IoT/RFID/sensors). NIST's 2023 competition selected ASCON (authenticated encryption + hashing). Other candidates: Grain, Trivium, PRESENT, SIMON, SPECK.

White-Box Cryptography: protects keys in software even under full adversary control of the execution environment (DRM, mobile payments); no provably secure construction exists — all practical schemes have been broken.

Steganography & Covert Channels: hiding messages in innocent-looking media (LSB embedding is simple but detectable; modern approaches use deep learning); covert channels repurpose non-data channels like timing or storage for communication.

12.2 Non-Malleability and Advanced Encryption/Signature Notions from Goldreich Vol. 2 (new in Ed. 5)

Beyond the IND-CPA/CCA1/CCA2 hierarchy of §2.5, Goldreich's treatment of encryption schemes names an additional, related security property directly:

Non-Malleable Encryption (NM-CPA/NM-CCA): semantic security (§2.5) guarantees an adversary learns nothing about a plaintext from its ciphertext, but says nothing about whether the adversary can transform a challenge ciphertext into a *different* ciphertext whose plaintext is meaningfully related to the original (e.g., "the same message plus one," or "the same message with a field flipped") without ever learning the original plaintext itself. Non-malleability closes this gap: no efficient adversary, given a challenge ciphertext, can produce a related ciphertext (or vector of ciphertexts) whose decryption is non-trivially related to the original plaintext, better than an adversary that never saw the challenge ciphertext at all. Non-malleability under chosen-ciphertext attack (NM-CCA2) is known to be equivalent to IND-CCA2, which is why the IND-CCA2 notion described in §2.5 is treated as "the" strong standard

notion in practice; the two properties (indistinguishability and non-malleability) were originally developed somewhat independently before this equivalence was established, and non-malleability remains the more intuitive way to describe the concrete failure that padding-oracle and other ciphertext-manipulation attacks (§14.3) exploit.

Universal One-Way Hash Functions (UOWHFs) and the One-Time-to-Many-Time Signature Paradigm: a UOWHF (also called a target-collision-resistant hash function) is weaker than a fully collision-resistant hash function (§4.1) — it only guarantees that, having committed to an input x in advance, an adversary cannot then find a colliding x' ≠ x, rather than guaranteeing no colliding pair can be found at all. This weaker, more efficiently achievable primitive is exactly what is needed to generically upgrade a one-time signature scheme (§9.5 — secure for signing only a single message per key) into a standard, many-time signature scheme: rather than directly hashing-and-signing with a collision-resistant function, the construction commits to each new one-time verification key using a UOWHF instance chosen fresh for that signature, so that an adversary who has not yet seen a given key has no advantage in finding a second key colliding under that instance. This UOWHF-based transform is a conceptually distinct route to the same "many-time signatures from one-time signatures" goal that the Merkle Signature Scheme's hash tree (§9.5) achieves via plain collision resistance, and is the construction Goldreich's Volume 2 uses to give collision-resistant hashing itself a signature-based application.

Unique Signatures: a signature scheme in which, for a given public key and message, at most one valid signature value exists (verification effectively fixes the signature, not merely accepts it) — useful for applications like verifiable random functions (VRFs) and lottery/randomness-beacon protocols (§12 above), where a party must not be able to bias an outcome by choosing among multiple valid signatures on the same input.

Strong (Super-Secure) Unforgeability: ordinary EU-CMA security (§2.5) only guarantees an adversary cannot forge a valid signature on a *new* message; it says nothing about whether an adversary who has seen one valid signature on a message can produce a *second, different* valid signature on that *same* message. Strong unforgeability closes this gap, additionally requiring that no efficient adversary can produce any new (message, signature) pair — including a new signature on an already-signed message — not obtained from the signer. This distinction matters for protocols (e.g., some token/coin-based e-cash constructions) that rely on each message having an essentially unique valid signature.

Incremental Signatures: a signature scheme designed so that, after a document is re-signed following a small, local edit (e.g., one paragraph changed in a large file), the new signature can be computed from the old signature and the edit alone, in time proportional to the size of the change rather than the size of the whole document — trading a more complex construction for large efficiency gains on frequently, incrementally updated documents.

13. Implementation Security and Side-Channel Resistance

13.1 Side-Channel Attacks

Exploit leakage from physical implementation, not mathematical weakness.

Timing attacks: exploit secret-dependent execution time; mitigated by constant-time code.

Cache-based attacks: Flush+Reload, Prime+Probe (cross-VM/process); Spectre/Meltdown 2018 exploit speculative/out-of-order execution; Foreshadow (L1TF) targets Intel SGX; ZombieLoad is another speculative-execution leak.

Power analysis: Simple Power Analysis (direct trace observation), Differential Power Analysis (statistical trace analysis); mitigated via randomization/masking/power-balancing.

Electromagnetic analysis: non-invasive emission measurement, effective against smart cards.

Acoustic cryptanalysis: recovering keys from sounds emitted during decryption (demonstrated against RSA).

Fault injection: laser fault injection, voltage/clock glitching, Rowhammer (memory bit-flips from repeated access), cold boot attacks (cooling RAM to retain contents after power loss).

13.2 Constant-Time Programming

No branches or memory lookups conditioned on secret data; no variable-time instructions (e.g., division) on secrets; use constant-time conditional move/select operations.

13.3 Memory Safety

Buffer overflows can leak keys or enable code execution; use-after-free can corrupt cryptographic state. Rust's memory safety (without garbage collection) makes it attractive for cryptographic code.

13.4 Formal Verification

Goal: prove implementations match mathematical specifications. Tools: Coq, Isabelle/HOL, F*, Cryptol*. Examples: HACL* (verified library in F*), Fiat-Crypto (verified ECC assembly), partially verified TLS 1.3 implementations.

13.5 Common Implementation Vulnerabilities

Nonce reuse (AES-GCM, ChaCha20-Poly1305): reveals the authentication key, enables forgery; XChaCha20-Poly1305's 192-bit nonce makes random generation statistically safe.

Invalid curve attacks: failing to validate input points lie on the curve reduces operations to small-order subgroups, leaking key material.

Low-entropy key generation: e.g., the 2008 Debian OpenSSL bug, ROCA; always use CSPRNGs.

API misuse: ECB mode, hardcoded keys, improper IVs; misuse-resistant designs (AES-GCM-SIV, NaCl/libsodium) help prevent these.

13.6 CSPRNGs

Required properties: unpredictability, forward secrecy, backtracking resistance. Sources: hardware RNGs (Intel RDRAND/RDSEED, TPM RNGs, quantum RNGs), OS CSPRNGs (/dev/urandom, getrandom(), CryptGenRandom/BCryptGenRandom), and research into chaotic maps (e.g., robust chaotic tent maps tested against NIST 800-22/TestU01).

Historical failures: Dual_EC_DRBG (suspected NSA-backdoored NIST RNG via suspicious parameters), Debian OpenSSL 2008 (entropy limited to process ID -> only 32,768 possible keys).

14. Cryptanalysis and Attack Methodologies

14.1 Classical Cryptanalysis

Frequency analysis: exploits non-uniform letter distributions; effective against simple substitution/transposition ciphers (Kerckhoffs' original technique — see §1.4 for his detailed column-based application against polyalphabetic ciphers, and §1.2.C for his worked attack on coded dictionaries).

Index of Coincidence (IC): statistical measure of letter-frequency distribution used to analyze polyalphabetic ciphers.

Known-plaintext attack: attacker has matching plaintext/ciphertext pairs.

Chosen-plaintext attack (CPA): attacker chooses plaintexts and obtains their ciphertexts — formalized as IND-CPA (§2.5).

Chosen-ciphertext attack (CCA): attacker chooses ciphertexts and obtains decryptions; CCA2 (adaptive) is the strongest practical model (§2.5).

14.2 Modern Cryptanalytic Techniques

Linear cryptanalysis: linear approximations of non-linear cipher components; needs large known-plaintext samples.

Differential cryptanalysis: studies how input differences propagate; highly effective against DES-era ciphers (and, historically, against early revisions of FEAL and LOKI — see §3.1).

Algebraic attacks: represent the cipher as polynomial equations, solved via Grobner bases, SAT solvers, or the XL algorithm.

Meet-in-the-middle attacks: build intermediate-value tables to attack double encryption — reduces 2DES's effective security from 2^112 to 2^57 (motivating 3DES).

Birthday attacks: exploit the birthday paradox to find collisions in O(2^(n/2)) instead of O(2^n) — sets the minimum secure hash output size.

Length-extension attacks: affect Merkle-Damgard hashes (MD5, SHA-1, SHA-2) — given H(m) and len(m), one can compute H(m || padding || m') without knowing m. Mitigation: HMAC, or a sponge construction (SHA-3).

14.3 Protocol-Level Attacks

Man-in-the-middle: intercept/modify communications; mitigated by certificate authentication, pinning, PSKs.

Replay attacks: retransmit valid captured messages; mitigated by nonces, timestamps, sequence numbers (see also the Needham–Schroeder ticket-replay weakness discussed in §7.6).

Padding oracle attacks: exploit invalid-padding error signals to decrypt ciphertext (affected SSL/TLS — POODLE, Lucky13); mitigated with AEAD and constant-time padding checks. These are, in essence, a practical exploitation of malleability — see the formal non-malleable-encryption notion in §12.2.

Downgrade attacks: force weaker protocol/cipher negotiation; TLS 1.3 removed version negotiation and added explicit downgrade protection.

14.4 Notable Historical Attacks

Attack	Target	Year	Mechanism
Dual_EC_DRBG	NIST RNG	2013	Suspected backdoor via parameter manipulation
Heartbleed	OpenSSL	2014	Buffer over-read leaking memory
POODLE	SSL 3.0	2014	Padding oracle in CBC mode
FREAK	TLS RSA_EXPORT	2015	Downgrade to 512-bit RSA
Logjam	TLS DH_EXPORT	2015	Downgrade to 512-bit DH
DROWN	SSLv2	2016	Cross-protocol attack via SSLv2
SWEET32	3DES/Blowfish	2016	Birthday attack on 64-bit blocks
ROCA	Infineon RSA	2017	Weak prime generation in TPMs

Efail	PGP/S-MIME	2018	Exfiltration via malleability gadgets
Spectre/Meltdown	CPUs	2018	Speculative-execution side-channels
Raccoon	TLS 1.2 DH	2020	Timing attack on DH key exchange

15. Blockchain and Distributed Systems Cryptography

15.1 Consensus Mechanisms

Proof of Work (PoW): miners solve hash puzzles (Bitcoin: SHA-256, Litecoin: Scrypt); high attack cost but energy-intensive; vulnerable to 51% attacks.

Proof of Stake (PoS): validators lock collateral (Ethereum moved to PoS in 2022, ~99% less energy); fast/efficient but risks stake centralization.

Delegated PoS (DPoS): token holders elect validating delegates (Cosmos, Tron, EOS); scalable but semi-centralized.

Practical Byzantine Fault Tolerance (pBFT): consensus once 2/3 of honest nodes agree (Hyperledger Fabric); efficient with immediate finality but not scalable beyond ~20 nodes; Sybil-vulnerable.

Proof of Authority (PoA): validators stake real-world identity/reputation (private/consortium chains); fast and accountable but not decentralized.

Others: Proof of Weight (Algorand), Proof of Importance (NEM), Proof of Capacity/Space (Chia, Filecoin).

15.2 Cryptographic Data Structures

Merkle Trees: hash-of-children tree enabling O(log n) inclusion proofs (Bitcoin, certificate transparency, Git).

Patricia (Radix) Tries: efficient key-value store used for Ethereum state.

Verkle Trees: vector-commitment alternative to Merkle-Patricia tries with smaller proofs; proposed for Ethereum scaling.

15.3 Cryptocurrency-Specific Primitives

Ring signatures: obscure the true signer among a ring (Monero).

Confidential transactions: hide amounts using Pedersen commitments + range proofs/Bulletproofs (Monero, Grin, Beam).

Multi-signature schemes: m-of-n signing; Schnorr MuSig aggregates into one signature; threshold signatures use distributed key generation.

MimbleWimble: combines confidential transactions with “cut-through” (no addresses, interactive transaction construction) — used by Grin and Beam.

15.4 Smart Contract Security

Reentrancy (recursive external calls draining funds — The DAO, 2016), integer overflow/underflow (largely mitigated in Solidity 0.8+), front-running (mempool transaction-ordering attacks), oracle manipulation (DeFi price-feed attacks).

16. Hardware Security and Trusted Computing

Hardware Security Modules (HSMs): dedicated key generation/storage/crypto-operation hardware; FIPS 140-2/140-3 validated; PCIe cards, network-attached, USB tokens; used for CA root keys, payment processing, code signing.

Trusted Platform Module (TPM): standardized secure crypto-processor (ISO/IEC 11889, TCG); secure key storage, attestation, measured boot; TPM 2.0 is current; ROCA (2017) affected Infineon TPMs.

Trusted Execution Environments (TEEs): - Intel SGX: encrypted memory enclaves isolated from OS/hypervisor; attacked by Foreshadow (L1TF), Plundervolt, SGAxe, Crosstalk; Intel is deprecating SGX for client CPUs. - ARM TrustZone: Secure World / Normal World split; widely used in mobile/IoT/payments. - AMD SEV: encrypts VM memory from the hypervisor; attacked by SEVered, CacheWarp. - Apple Secure Enclave: dedicated isolated processor handling biometrics, keys, secure boot.

Smart Cards: tamper-resistant embedded microprocessors (SIMs, EMV payment cards, ID cards); standards ISO/IEC 7816, EMVCo; vulnerable to DPA, fault injection, side-channel analysis.

Secure Elements: smaller tamper-resistant chips (hardware wallets like Ledger/Trezor, YubiKeys, IoT); typically Common Criteria EAL5+ certified for financial use.

17. Cryptographic Standards and Compliance

17.1 NIST Standards

FIPS 140-3 (replaced FIPS 140-2 in 2019): Security Levels 1-4, from software-only to tamper-resistant hardware; tested via CMVP.

FIPS 197 (AES), FIPS 180-4 (SHA-1/SHA-2), FIPS 202 (SHA-3), FIPS 198-1 (HMAC), FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA).

SP 800-38 series (block cipher modes), SP 800-56A/B (key establishment), SP 800-57 (key management), SP 800-90A/B/C (random number generation), SP 800-131A (algorithm transitions), SP 800-185 (SHA-3 derived functions: KMAC, TupleHash, ParallelHash).

17.2 Common Criteria (ISO/IEC 15408)

International security-certification standard; Evaluation Assurance Levels EAL1 (functionally tested) through EAL7 (formally verified, military/nuclear-grade); EAL4 = typical commercial software, EAL5 = smart cards/HSMs, EAL6 = high-security systems.

17.3 Other Standards

ISO/IEC 19790 (aligned with FIPS 140-3), PCI DSS (cardholder data encryption), HIPAA (protected health information encryption), GDPR (recommends encryption/pseudonymization).

17.4 PKI and Certificate Infrastructure

X.509 certificates (RFC 5280): subject, issuer, public key, validity, serial number, extensions; chains from Root CA -> Intermediate CA -> end entity.

Certificate Transparency (CT): Google-initiated public append-only certificate logs to catch misissuance.

OCSP: real-time revocation checking; OCSP Stapling improves privacy/performance.

DNSSEC: cryptographically signs DNS records (DNSKEY, RRSIG, DS); NSEC/NSEC3 prevent zone enumeration.

DANE: binds TLS certificates to DNS via TLSA records.

Key Management Lifecycle and Layering (new in Ed. 4). Beyond certificate-specific infrastructure, operational key management follows a broader lifecycle spanning generation, registration/certification, distribution, activation, use, expiry/deactivation, archival, and eventual destruction — with revocation (above) as an out-of-band interrupt to that lifecycle. Keys are also commonly layered by role rather than used flat: a top-level master key protects key-encrypting keys (KEKs), which in turn protect short-lived session/data keys actually used to encrypt bulk traffic — limiting the exposure from any single compromised key and matching key lifetime to how sensitive and how frequently-used each layer is. A Certificate Revocation List (CRL) is complemented in some PKI deployments by an Authority Revocation List (ARL), which specifically lists revoked CA (rather than end-entity) certificates. Key establishment more broadly splits into symmetric-key-based approaches using a trusted KDC (§7.6) and asymmetric/certificate-based approaches (this section) — the two historical lineages of the key-management problem.

17.5 Email Security

DKIM (cryptographic signatures on outgoing mail), DMARC (policy framework built on DKIM+SPF), SPF (authorized-sender DNS records), S/MIME and PGP (end-to-end email encryption — Efail (2018) exploited malleability gadgets in mail clients).

18. Socio-Political Dimensions and Cryptographic Ethics

18.1 The Political Nature of Cryptography

In “The Moral Character of Cryptographic Work,” Phillip Rogaway argues cryptography is inherently political, not value-neutral: because cryptographic primitives determine who can access, verify, or conceal information, cryptosystem design directly reconfigures power relations among citizens, corporations, and states. This echoes, in modern form, Kerckhoffs’ 1883 argument that a cipher’s design must not be a point of institutional vulnerability (see §1).

18.2 Surveillance and Privacy

Mass surveillance (automated data collection, traffic analysis, metadata mining) creates power asymmetries that chill speech and weaken democratic institutions. The cypherpunk movement responded with the principle that privacy should be enforced by code, not policy — exemplified by David Chaum’s blind signatures for untraceable digital cash and Phil Zimmermann’s PGP. Strong end-to-end encryption shifts enforcement power to end users.

18.3 Lawful Access Debates

Law enforcement often argues for mandatory key escrow, exceptional access, or client-side scanning. Security research shows that engineering any backdoor into an encrypted system fundamentally weakens it — the access mechanism itself becomes an exploitable attack surface.

The Clipper Chip / Skipjack episode (new in Ed. 4). The best-known historical case study in mandatory key escrow is the U.S. government’s early-1990s Clipper Chip initiative: a hardware encryption chip built around the classified Skipjack block cipher (later declassified) and standardized as the Escrowed Encryption Standard (EES), designed so that every encrypted call carried a Law Enforcement Access Field (LEAF) containing the session key encrypted under keys split between two government-designated escrow agents. The initiative was withdrawn from serious deployment after sustained cryptographic and public criticism, including a demonstrated flaw allowing the LEAF checksum to be forged — a concrete precedent frequently cited in the modern lawful-access debate as evidence that escrow mechanisms tend to introduce exactly the kind of exploitable structural weakness described above.

18.4 Technical Alternatives to Backdoors

Physical-seizure-constrained decryption: hardware rate-limiting requiring extended physical device possession plus a court-authorized secret — supports targeted forensic access while blocking automated mass remote surveillance.

Publicly traceable conditional decryption (witness encryption): every activation of a law-enforcement access capability is permanently, publicly logged, deterring abuse through transparency.

18.5 Export Controls and Human Rights

Wassenaar Arrangement: multilateral export-control regime historically restricting strong-encryption export.

Key disclosure laws: some jurisdictions compel decryption/password disclosure; “rubber-hose cryptanalysis” refers to coercion-based key extraction (not a technical attack) — deniable encryption is a technical countermeasure.

Human rights: UN Special Rapporteurs have affirmed encryption and anonymity as essential privacy protections underpinning other human rights.

18.6 Responsible Disclosure

Cryptographic vulnerabilities can have global impact; coordinated disclosure balances public safety against the need to patch (e.g., Heartbleed, ROCA, Dual_EC_DRBG were disclosed through coordinated — if imperfect — processes).

19. Authoritative Research Platforms and Learning Resources

19.1 Research Blogs

Platform / Author	Focus	Key Contributions
A Few Thoughts on Cryptographic Engineering (Matthew Green)	Protocol flaws, ZKPs, hardware backdoors, provable security	ROM breakdowns, ZKP primers, Dual_EC_DRBG post-mortems
ImperialViolet (Adam Langley)	TLS/SSL internals, ECC, memory safety	X.509 parsing vulnerabilities, constant-time routines, TLS 1.3 optimization
Schneier on Security (Bruce Schneier)	Foundational theory, security architecture, policy	System threat models, Kerckhoffs-compliant advocacy
Trail of Bits Research	Code audits, ZKP circuits, PQC, side-channels	Static analysis tools, ZK constraint audits, compiler-level flaw detection
Cloudflare Research Blog	Applied PQC, OHTTP, TLS, ECC, distributed randomness	ECDSA vs EdDSA guides, PQ KEM deployment, drand beacons
David Wong’s Cryptography Blog	ECC arithmetic, MPC, Noise, ZK-SNARKs	Mathematical breakdowns of applied proof-systems
NCC Group Research Blog	Cryptanalysis, protocol bugs, hardware attacks	Side-channel leakage identification, non-standard curve cryptanalysis

19.2 Video / Course Resources

Dan Boneh — Stanford CS255 / Online Cryptography: perfect secrecy proofs, provable security reductions, number theory.

Christof Paar — Introduction to Cryptography: LFSRs, Trivium, ARX ciphers, Galois field arithmetic, DPA. (See also §22 for Paar, Pelzl & Güneysu’s companion textbook, Understanding Cryptography, reviewed at the topic level in this edition.)

Silvio Micali — MIT 6.875 Foundations of Cryptography: OWFs, Goldreich-Levin, Blum-Micali PRGs.

Yael Kalai — MIT 6.5630 Advanced Topics: interactive proofs, sum-check protocol, GKR protocol.

Brynmor Chapman — MIT 6.1200J Lecture 10: Euclidean/Extended Euclidean algorithms, modular exponentiation.

UC Berkeley RDI ZK MOOC (Boneh, Thaler, Zhang, Goldwasser, Song): R1CS/AIR/PlonKish, KZG/IPA, recursive SNARKs.

ZK Whiteboard Sessions (ZK Hack/Polygon): Plonky2, FRI, prover acceleration, lattice-based PQ SNARKs.

Alfred Menezes — Mathematics of Lattice-Based Cryptography: lattice geometry, LLL, SVP/SIVP/CVP, M-LWE/M-SIS.

Chris Peikert & Oded Regev — Advanced lattice seminars: LWE formulation, lattice trapdoors, Gaussian sampling.

Chalk Talk PQ Series: Shor's algorithm/QFT mechanics, LWE noise-flooding visualization.

FHE.org / Zama seminars (Paillier, Gentry, Benamira): TFHE, bootstrapping, BGV/BFV/CKKS, TT-TFHE.

Computerphile / Andrea Corbellini: elliptic curve visualizations, ECDSA mechanics.

IACR Archive (@TheIACR): 4,100+ recorded talks from Eurocrypt, Crypto, Asiacrypt, CHES, Real World Crypto.

20. Strategic Outlook and Future Directions

Post-quantum migration: Canada's roadmap targets initial migration plans by April 2026, priority transitions by 2031, full migration by 2035; CISA released an initial PQC-supporting hardware/software category list (January 2026). Organizations should start with cryptographic inventory and risk assessment, then migrate the most vulnerable systems first.

Privacy-preserving convergence: combining FHE, MPC, and ZKPs enables computation verifiably correct without revealing inputs or intermediate results — a defense-in-depth approach for sensitive AI applications.

Formal verification and correctness: ongoing focus on formally verified crypto code and misuse-resistant primitive designs to close the gap between mathematical proofs and real-world implementation bugs.

Cryptographic agility: the ability to swap algorithms quickly matters for responding to cryptanalytic breaks, migrating to PQC, and meeting evolving standards — via regular key rotation and protocols that support secure algorithm negotiation with downgrade protection.

The persistent operational gap: mathematical security proofs (§2.5) routinely fail to account for implementation-level side channels, low-entropy key derivation, and API misuse — driving continued investment in formal verification, misuse-resistant design, trust-minimizing ZK paradigms, constant-time implementations, and security auditing.

Final synthesis: cryptography sits at the intersection of mathematics, engineering, and social power. Its trajectory depends on mathematical rigor, engineering excellence, standardization, ethical commitment to privacy, quantum readiness, privacy-preserving computation, and decentralization of trust (MPC, threshold cryptography, blockchain) — the same tension between provable strength and practical, accountable deployment that Kerckhoffs first articulated in 1883 (§1).

21. Glossary of Key Terms

Term	Meaning
AEAD	Authenticated Encryption with Associated Data
CCA / CPA	Chosen-Ciphertext / Chosen-Plaintext Attack
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
DH / DLP	Diffie-Hellman / Discrete Logarithm Problem
EU-CMA	Existential Unforgeability under Chosen-Message Attack (signature/MAC security)
ECC / ECDH / ECDSA	Elliptic Curve Cryptography / Diffie-Hellman / Digital Signature Algorithm
FHE	Fully Homomorphic Encryption
HMAC	Hash-based Message Authentication Code
IBE	Identity-Based Encryption
IND-CPA / IND-CCA	Indistinguishability under Chosen-Plaintext / Chosen-Ciphertext Attack
IV	Initialization Vector
KDF / KEM	Key Derivation Function / Key Encapsulation Mechanism
LWE	Learning With Errors
MAC / MPC	Message Authentication Code / Multi-Party Computation
NIST	National Institute of Standards and Technology
NM-CPA / NM-CCA	Non-Malleability under Chosen-Plaintext / Chosen-Ciphertext Attack (new in Ed. 5)
Nonce	Number used once
OWF	One-Way Function
PPT	Probabilistic Polynomial-Time (the standard model of an “efficient” adversary)
PQC	Post-Quantum Cryptography
PRF / PRG / PRP	Pseudo-Random Function / Generator / Permutation
QKD	Quantum Key Distribution
ROM	Random Oracle Model
RSA	Rivest-Shamir-Adleman
SIS	Short Integer Solution
SNARK / STARK	Succinct / Scalable (Transparent) Argument of Knowledge
TEE	Trusted Execution Environment
TLS	Transport Layer Security
UOWHF	Universal One-Way Hash Function / target-collision-resistant hash function (new in Ed. 5)
WI	Witness Indistinguishability
ZKP	Zero-Knowledge Proof

22. Recommended Reading and References

Foundational texts: Katz & Lindell, Introduction to Modern Cryptography (2nd ed.) — the primary source for the formal-definitions/provable-security framework in §2.5; its four-part structure (classical ciphers -> private-key crypto -> number theory -> public-key crypto, plus

appendices on mathematical background and algorithmic number theory) is a natural next step for readers who want the full proofs behind the summaries in §2-§6 above. (This edition's compiler could not directly re-verify this book's content — see the Update note at the top of this document.) Also: Goldreich, Foundations of Cryptography (Vols. 1-2) — both volumes have now been supplied and reviewed at the topic-structure level (Volume 1, "Basic Tools," in Edition 4; Volume 2, "Basic Applications," covering Encryption Schemes, Digital Signatures and Message Authentication, and General Cryptographic Protocols, in this edition — see Update Note); Boneh & Shoup, A Graduate Course in Applied Cryptography; Smart, Cryptography Made Simple; Paar, Pelzl & Güneysu, Understanding Cryptography: From Established Symmetric and Asymmetric Ciphers to Post-Quantum Algorithms (2nd ed.) — supplied and reviewed at the topic-structure level in this edition (see Update Note); its practical, engineering-oriented four-teen-chapter structure (symmetric ciphers through PQC and key management) is a strong complement to the more formal, definition-first treatment summarized in §2 and §6-§9 above; Menezes, van Oorschot & Vanstone, Handbook of Applied Cryptography — likewise supplied and reviewed at the topic-structure level in Edition 4; still the standard applied reference for detailed algorithmic descriptions, worked pseudocode, and exhaustive citations across every primitive summarized in §2-§17 above.

Standards: NIST FIPS 140-3, FIPS 197, FIPS 203-205; RFC 8446 (TLS 1.3); RFC 8439 (ChaCha20-Poly1305).

Key papers: Goldwasser, Micali & Rackoff, "The Knowledge Complexity of Interactive Proof Systems" (1985); Bellare & Rogaway, "Random Oracles are Practical" (1993, formalizes the ROM of §2.6); Canetti, Goldreich & Halevi, "The Random Oracle Methodology, Revisited" (1998, the ROM impossibility result of §2.6); Gentry, "Fully Homomorphic Encryption Using Ideal Lattices" (2009); Ben-Sasson et al., "Scalable, transparent, and post-quantum secure computational integrity" (2018); ElGamal, "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms" (1985, the source of both the encryption and signature schemes in §6.2 and §6.4, new in Ed. 5); Needham & Schroeder, "Using Encryption for Authentication in Large Networks of Computers" (1978, the source of the protocol in §7.6, new in Ed. 5).

Historical primary source: Kerckhoffs, A. "La Cryptographie Militaire," Journal des Sciences Militaires, Vol. IX (Jan–Feb 1883) — origin of Kerckhoffs' Principle (§1 above). Both parts have now been consulted directly as scanned originals for this compilation:

- Part I (pp. 5–38): http://petitcolas.net/kerckhoffs/crypto_militaire_1.pdf
- Part II (pp. 161–191): http://petitcolas.net/kerckhoffs/crypto_militaire_2.pdf

Online resources: Cryptography Engineering Blog (Matthew Green), ImperialViolet (Adam Langley), Schneier on Security (Bruce Schneier), Trail of Bits Research Blog, IACR ePrint Archive, NIST Computer Security Resource Center.